

SOLPS-ITER simplified heat transfer model for plasma facing components

By

Stefano Carli

stefano.carli@polito.it

stefano.carli@live.com

NEMO Group, Dipartimento Energia, Politecnico di Torino, Torino, Italy

Last update: June 29, 2017

Contents

1. Introduction.....	1
1.1 Overview of the ITER divertor cooling configuration	1
1.2 FreeFEM++	7
2. Simplified heat transfer model.....	8
2.1 Vertical targets.....	8
2.1.1 Geometry	13
2.1.2 Convective heat transfer coefficient	14
2.1.3 Benchmark	20
2.2 Reflector plates	25
2.2.1 Geometry	26
2.2.2 Heat transfer coefficient.....	28
3. How to use it	32
3.1 Definition of plot zones in DG	32
3.2 Formatted input files.....	33
3.2.1 Monoblock input: b2ymb.dat.....	33
3.2.2 Reflector plate input: b2yrp.dat	35
3.2 Executing the thermal analysis	38
3.3 Standard output.....	39
3.4 Windows/iOS users	40
4. References.....	42
5. Material properties	44
6. Script of the monoblock heat transfer model	49

7. Script of the reflector plates heat transfer model	57
---	----

List of Figures

Figure 1: current ITER divertor design (reproduced from [1]).	2
Figure 2: current ITER divertor design (reproduced from [2]).	3
Figure 3: cross section of the RP or DOME plasma facing unit (reproduced from [3]).	3
Figure 4: view of the monoblocks arrangement (reproduced from [2]).	4
Figure 5: cross section of the divertor monoblocks with poloidal chamfer (reproduced from [2]).	5
Figure 6: vertical cross section of the plasma chamber (reproduced from [2]).	5
Figure 7: ITER divertor coolant flow path (reproduced from [3]).	6
Figure 8: radial-poloidal plane with SOLPS-ITER plasma domain (light gray) and plasma facing units.	9
Figure 9: the 3D conduction and advection-diffusion problem of the monoblock is split into a series of 2D conduction problems on the monoblock cross section, along the poloidal direction, coupled with an energy conservation equation for the coolant.	10
Figure 10: scheme of the simplified thermal model of the monoblock along the coolant flow path (i.e. the poloidal direction). The heat flux q from SOLPS-ITER results is represented with red arrows, the W solid in light gray, the Cu interlayer and CuCrZr tube in orange and the coolant in blue. The blue arrows indicate the direction of the fluid flow.	10
Figure 11: 2D cross section of the monoblock along poloidal direction with description of the boundary conditions applied.	12
Figure 12: 2D cross section of the monoblock along poloidal direction with description of the geometrical quantities.	13
Figure 13: cross section of the monoblock cooling configuration with twisted tape.	15
Figure 14: parametric plot of the heat transfer coefficient for the monoblocks.	17
Figure 15: parametric plot of the heat transfer coefficient for the monoblocks with the polynomial fit on the lower temperature range.	18

Figure 16: parametric plot of the heat transfer coefficient for the monoblocks with the polynomial fit on the upper temperature range.....	19
Figure 17: results of the benchmark between the FreeFEM++ heat transfer model and the ANSYS model for a W armor of 6 mm (a) and 8 mm (b). The solid lines in red (blue) refer to the FreeFEM++ values for the corner (center) of the top surface, while the dashed lines in magenta (cyan) refer to the ANSYS values for the corner (center) of the top surface.	21
Figure 18: result of the benchmark between the ANSYS (a) and FreeFEM++ (b) heat transfer models for the case of W armor of 6 mm and a heat flux of 20 MW/m ²	23
Figure 19: 2D cross section of the hypervapotron along poloidal direction with description of the boundary conditions applied.	25
Figure 20: 2D cross section of the hypervapotron along poloidal direction with description of the geometrical quantities.	26
Figure 21: cross section of the hypervapotron cooling configuration.....	28
Figure 22: parametric plot of the heat transfer coefficient for the hypervapotron.	29
Figure 23: parametric plot of the heat transfer coefficient for the hypervapotron with the polynomial fit on the lower temperature range.....	30
Figure 24: parametric plot of the heat transfer coefficient for the hypervapotron with the polynomial fit on the upper temperature range.....	31
Figure 25: Picture of the Plot Zone definition mask in DG.	33

List of Tables

Table 1:geometrical data for the monoblock heat transfer model.....	14
Table 2: results of the benchmark between the FreeFEM++ heat transfer model and the ANSYS model.....	22
Table 3: results of the benchmark between the FreeFEM++ heat transfer model and the Code Aster in terms of W top surface corner and centerline temperature.	24
Table 4: results of the benchmark between the FreeFEM++ heat transfer model and the Code Aster model in terms of maximum Cu interlayer and CuCrZr pipe temperatures.....	24
Table 5:geometrical data for the hypervapotron heat transfer model.	27
Table 6: Material properties of CuCrZr [11].....	44
Table 7: Material properties of Cu [11].	45
Table 8: Material properties of W [11].	46
Table 9: Material properties of 316L SS [16].	47

Chapter 1

Introduction

1.1 Overview of the ITER divertor cooling configuration

The current configuration of the ITER divertor is shown in Figure 1 [1]. The main components are:

- The Vertical Targets (VTs), made of tungsten (W) monoblocks, which allow the spreading of the incident plasma power flux;
- The Reflector Plates (RPs), made of W flat tiles bonded to copper-chromium-zirconium (CuCrZr) heat sink with hypervapotron cooling technology, which protect the Cassette Body (CB) from plasma radiation and strike point (SP) movement;
- The dome, made of W flat tiles bonded to CuCrZr heat sink with hypervapotron cooling technology, which aims at improving the neutral confinement and acts as a neutron screen protecting the diagnostics installed under it.

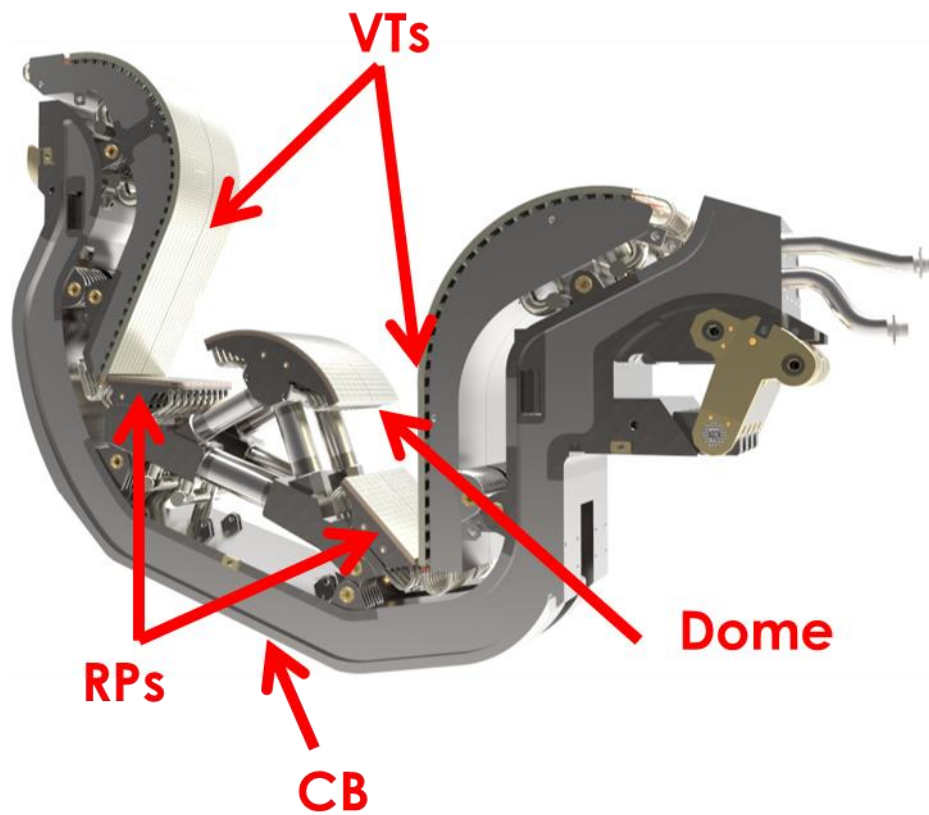


Figure 1: current ITER divertor design (reproduced from [1]).

The W monoblocks configuration of the VTs is shown in Figure 2: the CuCrZr tube is bonded to the W tiles through an interlayer of copper. In the inside of the tube a swirl tape, made of CuCrZr, helps in enhancing the heat transfer between the coolant (subcooled water) and the solid.

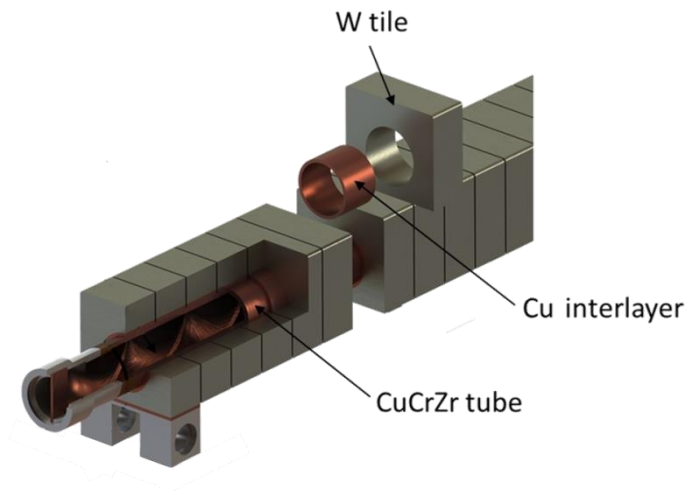


Figure 2: current ITER divertor design (reproduced from [2]).

The RPs and dome configuration is shown in Figure 3: the support body is made of 316L stainless steel (SS) while the dented hypervapotron top surface is made of CuCrZr. A Cu layer joins the plasma facing surface, made of W, with the heat sink.

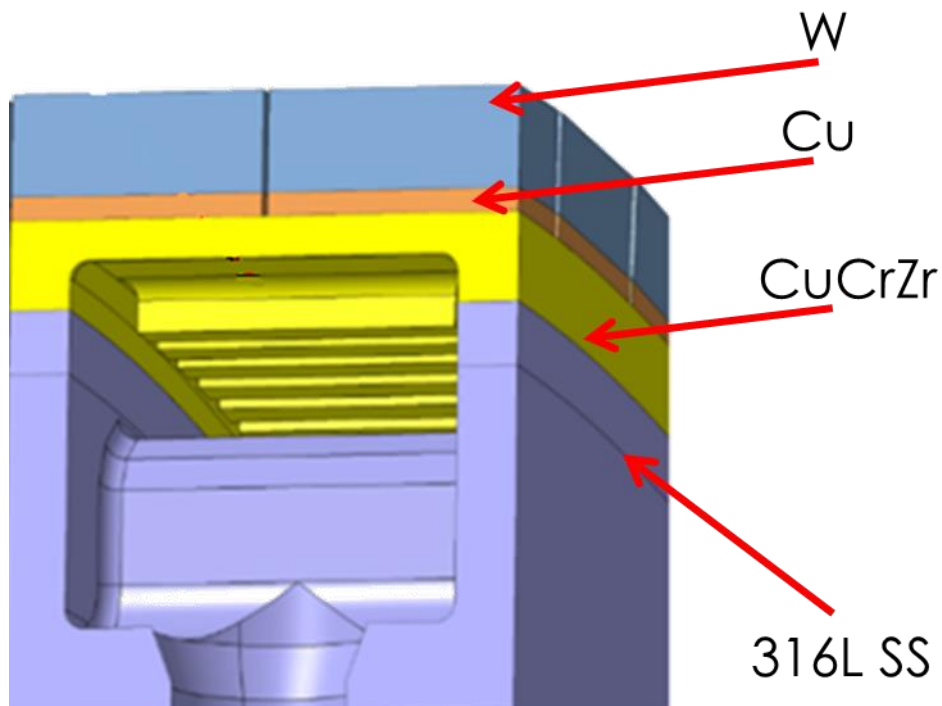


Figure 3: cross section of the RP or DOME plasma facing unit (reproduced from [3]).

The tiles of the VTs, RPs and dome are not perfectly rectangular on the toroidal and poloidal cross section because of melting issues on the leading edges of the W tiles, separated by small gaps, see Figure 4.

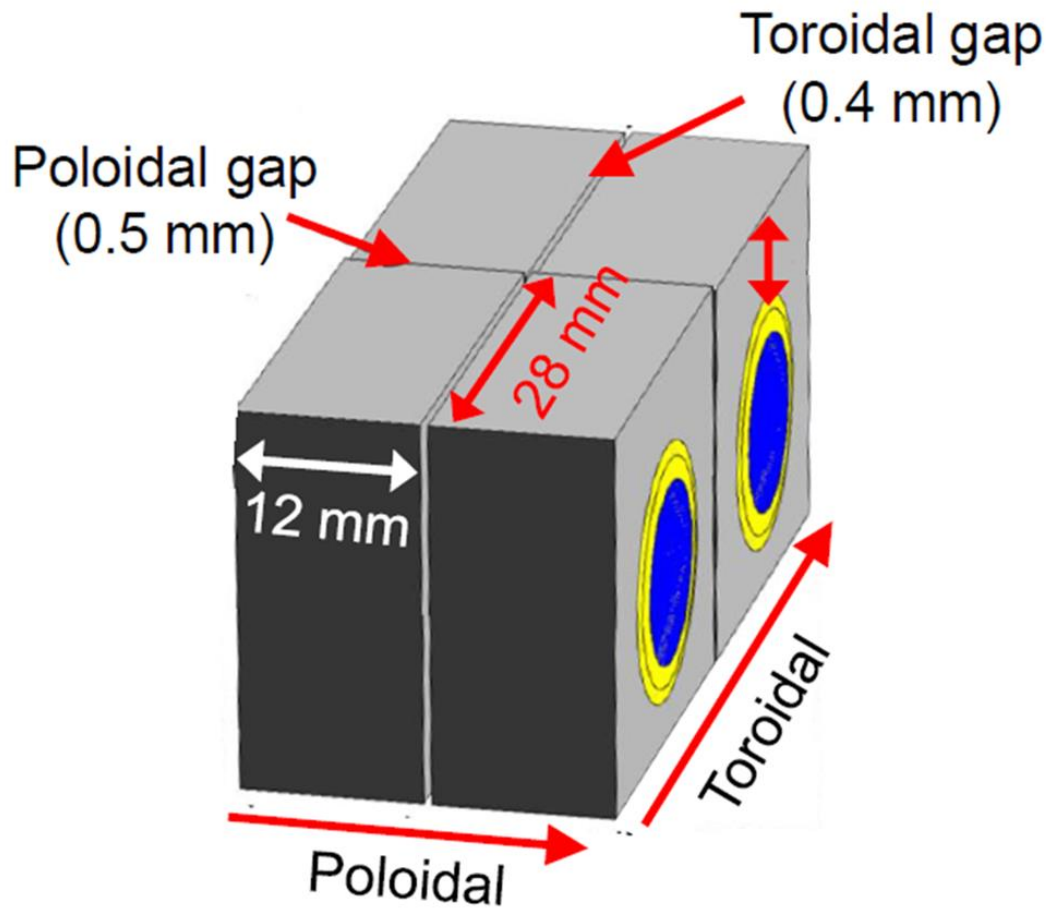


Figure 4: view of the monoblocks arrangement (reproduced from [2]).

Therefore, a poloidal and toroidal chamfer is applied, see e.g. Figure 5 for the poloidal one, in order to shadow the leading edges, with the drawback of increasing the magnetic field line strike angle and therefore the heat flux on the plasma wetted surface.

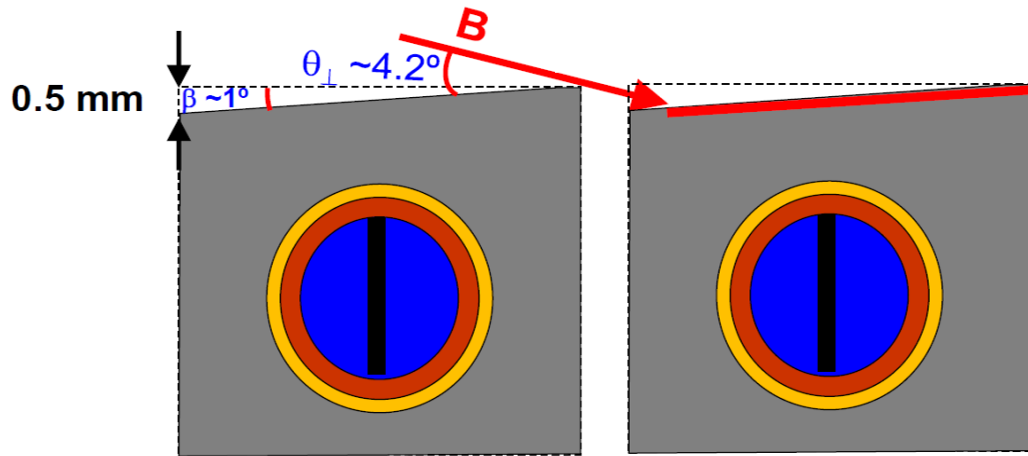


Figure 5: cross section of the divertor monoblocks with poloidal chamfer (reproduced from [2]).

Additionally, the divertor targets on each cassette are tilted by an angle $\alpha \sim 0.5^\circ$ [4] to avoid the melting of the leading edges between one cassette and the neighboring ones, see Figure 6.

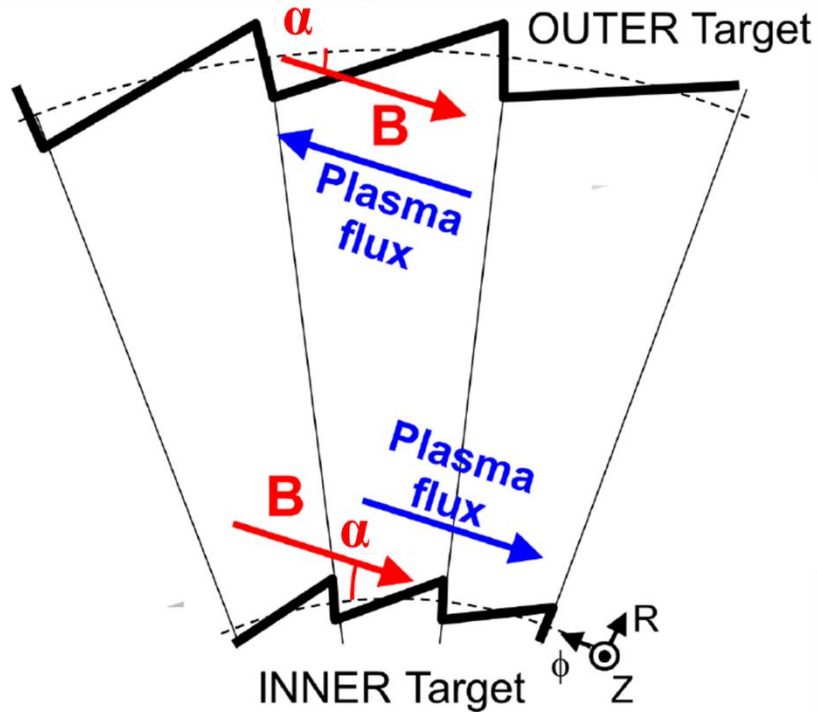


Figure 6: vertical cross section of the plasma chamber (reproduced from [2]).

The flow path followed by the coolant inside the divertor channels is reported in Figure 7. Entering the cassette from the outboard (right side in the figure), the coolant is fed in parallel to the monoblocks of the outer VT from the V-shaped corner formed with the RP (as the nominal plasma equilibrium puts the SP, where the heat load is larger, a few centimeters above the corner) and flows poloidally to the top of the outer divertor baffle (point 3 in Figure 7). It then cools the lower part of the inner VT, starting again from the corner to the baffle (point 5). The outflow of the inner monoblocks is then fed to the inner RP hypervapotron channels, with the inlet near the corner with the VT and the outlet towards the dome, which is the subsequent cooled component together with its supporting structures. Finally, the outlet of the dome is connected to the outer RP having a flow direction from the dome towards the outer VT.

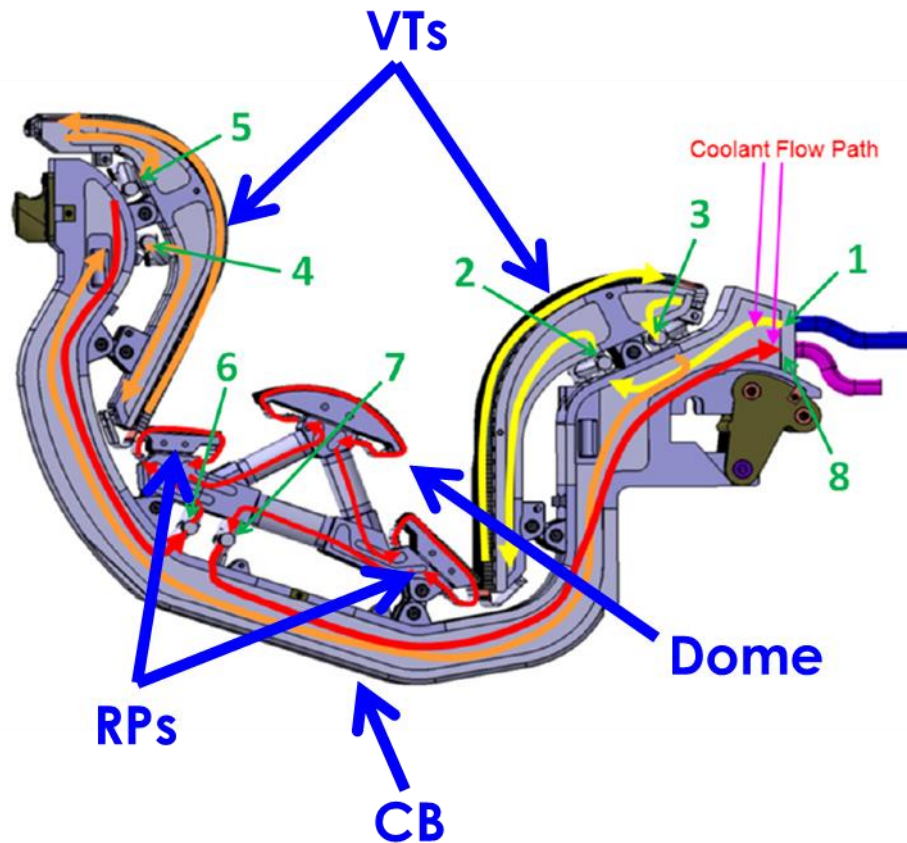


Figure 7: ITER divertor coolant flow path (reproduced from [3]).

Different studies regarding the ITER divertor and the cooling capabilities of the monoblock and hypervapotron technologies are available, some of them can be found in [5, 6, 7, 8] and references therein.

1.2 FreeFEM++

FreeFEM++ [9] is a free, open-source software package for 2D / 3D finite element computations which enables to solve partial differential equations relatively easily. It is written in C++ and much of the syntax is similar to that of C++. The package allows the mesh generation (both structured and unstructured), includes a wide range of finite elements and integrated linear solvers (e.g. CG, GMRES, UMFPACK) and visualization tools. For detailed information on the software and its installation procedure please refer to the website <http://www.freefem.org/> and the user manual in the FreeFem++ installation directory.

Chapter 2

Simplified heat transfer model

2.1 Vertical targets

The results of SOLPS-ITER simulations allow a detailed description of the heat load in W/m^2 on the desired wall surfaces (see plot zones in section 3.1 and in SOLPS-ITER manual [10]), both for plasma wetted areas and not. The heat load profiles are given using a postprocessing routine within SOLPS-ITER, see section 3.2, which provides the heat flux along the poloidal direction of the walls, which means the flow path followed by the coolant along, in order, the outer VT, the inner VT, the inner RP, the dome and the outer RP, see Figure 8.

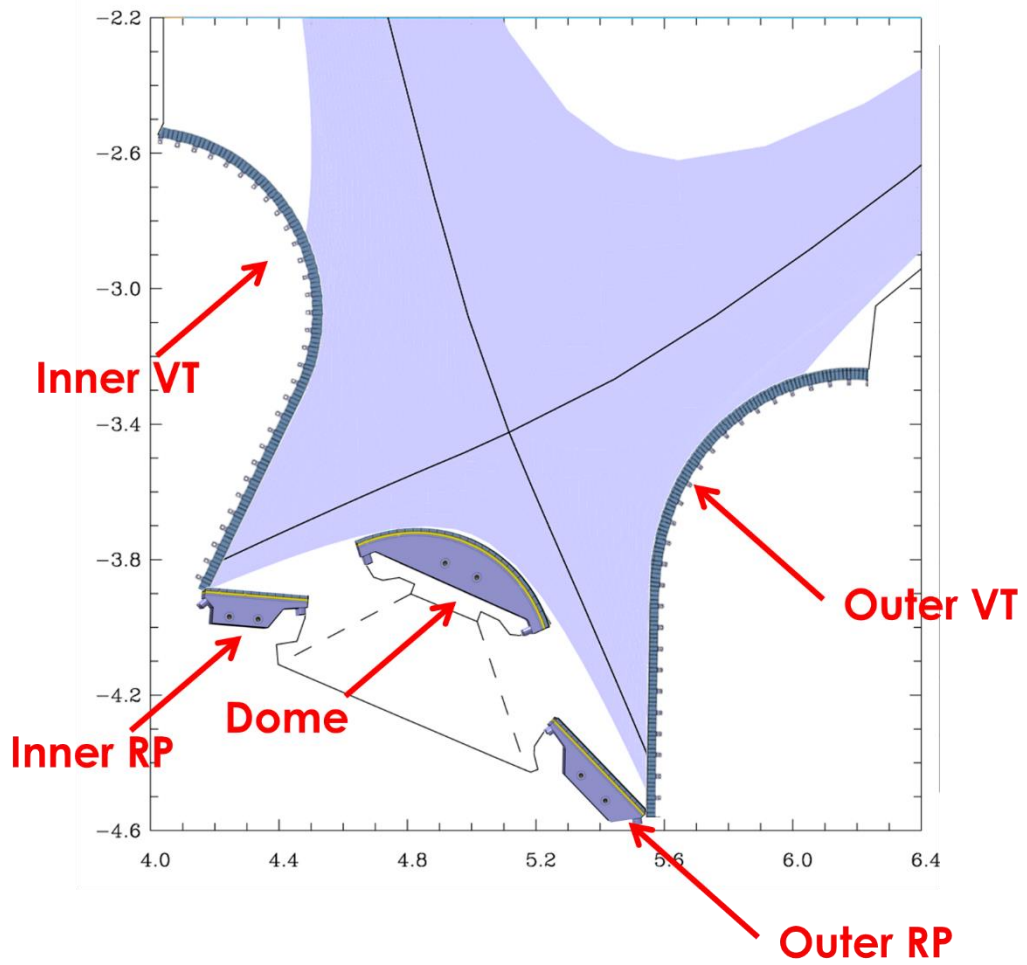


Figure 8: radial-poloidal plane with SOLPS-ITER plasma domain (light gray) and plasma facing units.

The first simplifying assumption made for the development of the heat transfer model is to split the 3D nature of the problem into a series of 2D problems on the monoblock cross section, as in Figure 5, along the poloidal direction, see Figure 9 and Figure 10, on which the heat diffusion in the solids is studied and the presence of the coolant is considered only as a boundary condition for the 2D conduction problem, *i.e.* no advection-diffusion computation is carried out on the fluid domain. In fact, for what concerns the fluid, a simple energy conservation equation is applied between adjacent 2D cross sections along the poloidal direction, assuming a constant mass flow rate and specific heat capacity, see Figure 10 and Eq. (1).

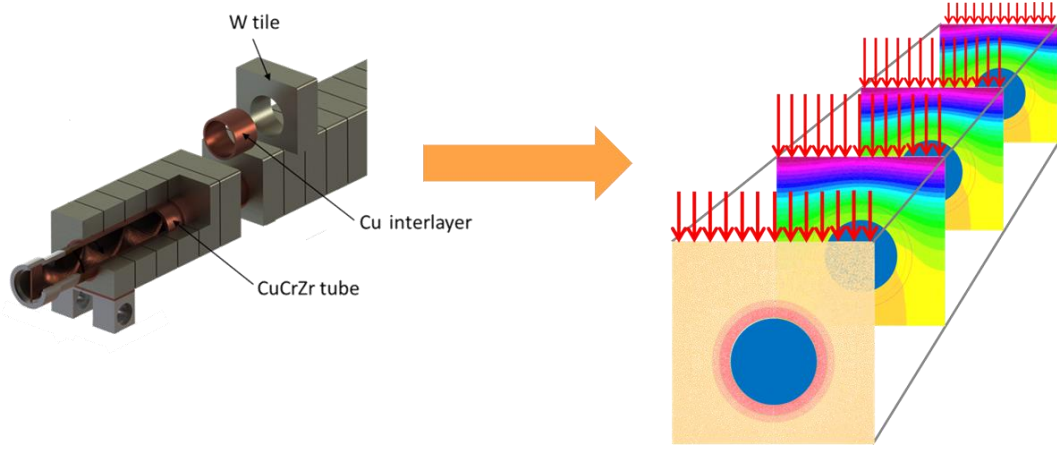


Figure 9: the 3D conduction and advection-diffusion problem of the monoblock is split into a series of 2D conduction problems on the monoblock cross section, along the poloidal direction, coupled with an energy conservation equation for the coolant.

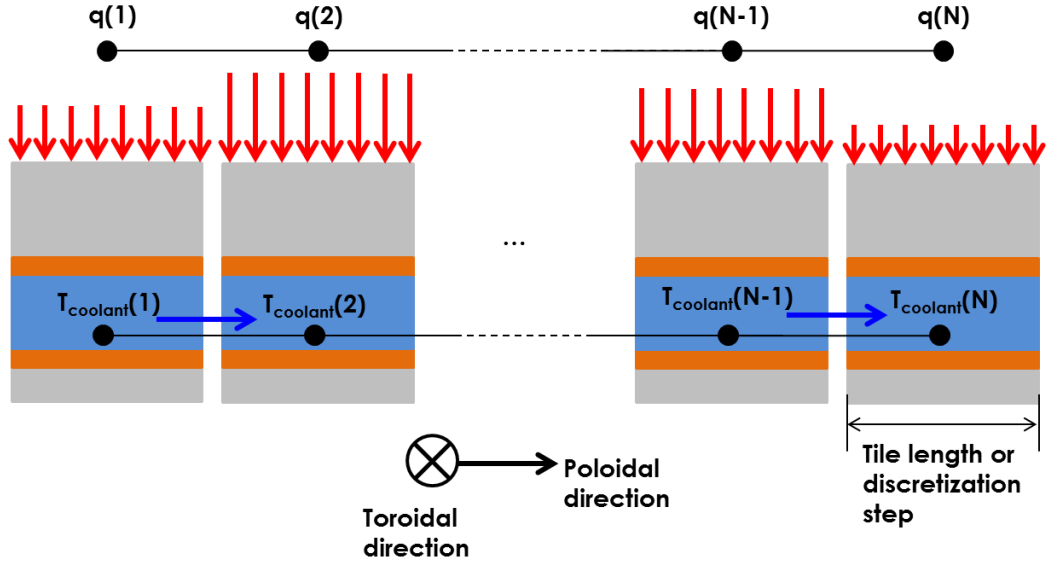


Figure 10: scheme of the simplified thermal model of the monoblock along the coolant flow path (*i.e.* the poloidal direction). The heat flux q from SOLPS-ITER results is represented with red arrows, the W solid in light gray, the Cu interlayer and CuCrZr tube in orange and the coolant in blue. The blue arrows indicate the direction of the fluid flow.

$$T_{coolant}(i) = T_{coolant}(i-1) + q(i) \cdot \frac{A_{top}(i)}{c_p \cdot \dot{m}} \quad (1)$$

In Eq. (1), $T_{coolant}$ is the coolant temperature, i is the discretized coordinate along the poloidal direction, q is the heat flux, A_{top} is area of the top surface subject to the heat load, c_p is the coolant specific heat capacity and $\dot{m}$ is the coolant mass flow rate.

It must be noted that the heat transfer along the poloidal direction in the CuCrZr tube and Cu interlayer has been neglected, meaning that the only coupling between one discretization node and the other is the fluid temperature. The discretization step between each 2D cross section of the monoblock along the poloidal direction could be chosen as the tile poloidal length or any desired discretization step.

As the 2D conduction problems are concerned, Figure 11, the top surface of the monoblock experiences an imposed heat flux boundary condition, while the other outer edges are modeled conservatively with an adiabatic boundary condition, which means that there is no heat transfer between monoblocks along the toroidal direction nor with the supporting structures below, condition that, together with the adiabaticity between the tiles in the poloidal direction described before, gives an upper estimate of the temperature profile. On the inner side of the CuCrZr pipe, a Robin boundary condition is applied, with a constant coolant temperature and a convective heat transfer coefficient described in section 2.1.3. When solving the conduction problem on each cross section, temperature dependent material properties are taken into account, see Appendix 1.

As shown in Figure 10, the heat flux q is supposed to be constant on each discretized poloidal length, without taking into account any poloidal chamfer of the monoblock's tiles. On the other hand, looking at the 2D cross section along the poloidal length, see Figure 5, the heat flux q is arranged in such a way that on the top surface portion shadowed by the toroidally neighboring tile only the contribution from radiation and neutral particles is taken into account, while on the plasma wetted portion, identified with the red line in Figure 5, also the contribution to the heat flux coming from the plasma is accounted for, corrected by the strike angle difference given by the toroidal chamfer β as well as the VT tilting angle α , see Figure 6. Therefore, the heat flux q on the top surface of each 2D cross section is given by:

$$q = \begin{cases} q_{neutrals} + q_{radiation} & \text{on shadowed area} \\ \frac{q_{plasma}}{\cos(\alpha + \beta)} + q_{neutrals} + q_{radiation} & \text{on plasma wetted area} \end{cases} \quad (2)$$

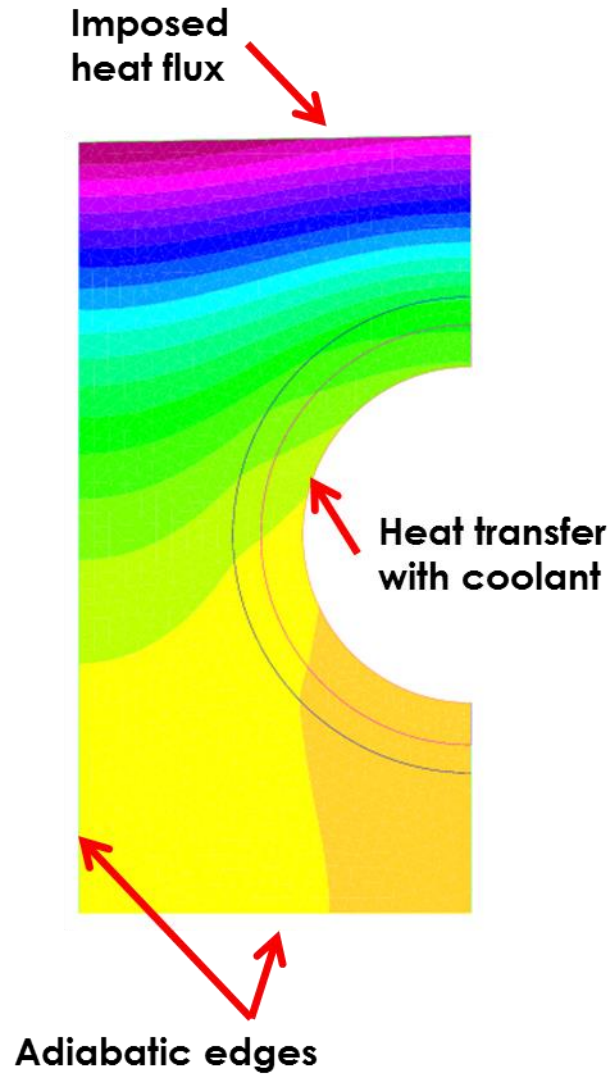


Figure 11: 2D cross section of the monoblock along poloidal direction with description of the boundary conditions applied.

2.1.1 Geometry

The geometrical data for the monoblock cross section along the poloidal direction are shown in Figure 12 and listed in Table 1 [11]. Note that each of the quantities reported can be modified by the user through the ad-hoc input file described in section 3.2.1.

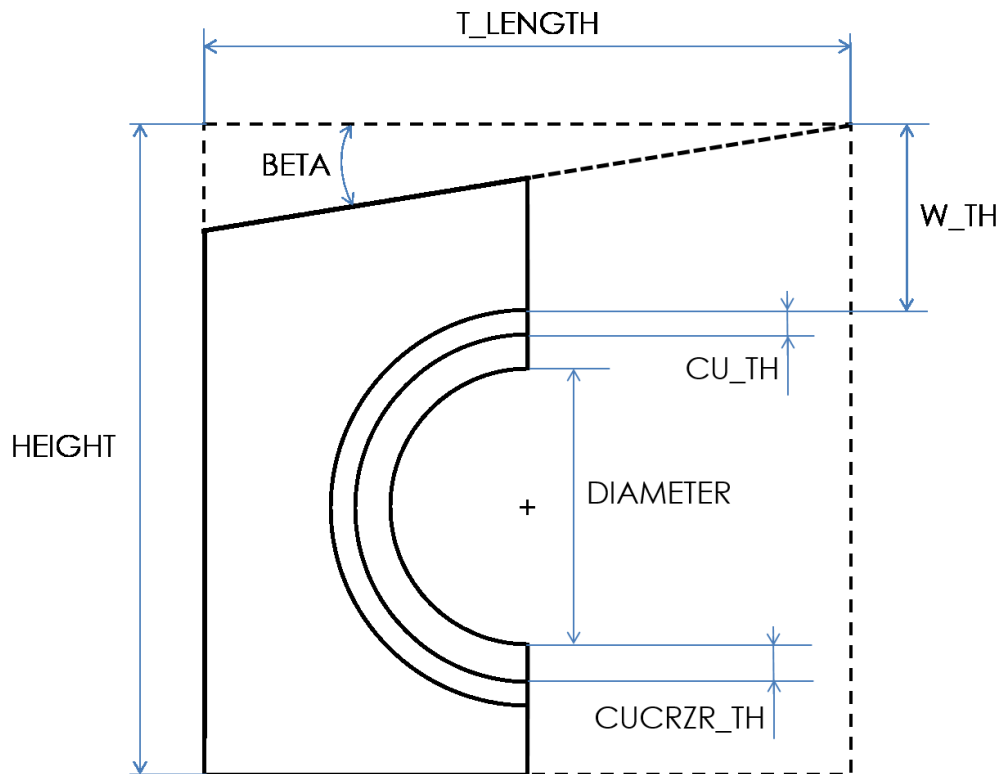


Figure 12: 2D cross section of the monoblock along poloidal direction with description of the geometrical quantities.

Table 1: geometrical data for the monoblock heat transfer model.

Quantity	Description	Value
HEIGHT	Tile height	28 mm
T_LENGTH	Tile toroidal length	28 mm
CU_TH	Thickness of the Cu interlayer	1 mm
CUCRZR_TH	Thickness of the CuCrZr pipe	1.5 mm
DIAMETER	Inner diameter of the CuCrZr pipe	12 mm
W_TH	Thickness of the W armor	6 mm
P_LENGTH	Tile poloidal length / poloidal discretization step	12 mm
T_GAP	Toroidal gap between adjacent monoblocks	0.5 mm
ALPHA	Target tilting angle	1.0°
BETA	Toroidal chamfer angle	1.0°
THETA	Toroidal strike angle of the magnetic field lines	4.2°

2.1.2 Convective heat transfer coefficient

The expression for the convective heat transfer coefficient adopted is that in [12], which makes use of two different correlations, one for wall temperatures lower than the onset of nucleate boiling (ONB) and one for subcooled boiling (note that the model does not support nor takes into account the detailed boiling phenomena, it just uses a coolant bulk temperature to evaluate the heat transfer coefficient).

If the wall temperature is lower than the ONB point, the heat transfer coefficient h (W/m²/K) is evaluated using the Sieder-Tate correlation:

$$h_{ST} = 0.027 \cdot f \cdot Re^{0.8} \cdot Pr^{0.33} \cdot \left(\frac{\mu_b}{\mu_w} \right)^{0.14} \cdot \frac{k_b}{D_h} \quad (3)$$

in which f is the fin effect factor and is set to 1.15 for the twisted tape configuration, Re is the Reynolds number, Pr is the Prandtl number, μ is the coolant dynamic viscosity (Pa s) at bulk temperature (subscript b) and at wall temperature (subscript w), k is the coolant thermal conductivity (W/m/K) at bulk temperature and D_h is the hydraulic diameter (adopted also for the Re calculation) and defined as:

$$D_h = 4 \cdot \frac{\left(\frac{\pi D^2}{4} - \delta \cdot D\right)}{\pi D + 2D - 2\delta} \quad (4)$$

in which D is the pipe inner diameter and δ is the twisted tape thickness (0.8 mm for inner VT and 2 mm for outer VT) as in Figure 13.

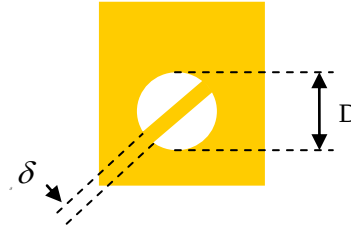


Figure 13: cross section of the monoblock cooling configuration with twisted tape.

The twisted tape also affects the velocity at which the Re number should be calculated. The formula to be used is:

$$V = V_L \left(1 + \frac{\pi^2}{4r^2}\right)^{0.5} \quad (5)$$

where V_L is the coolant longitudinal speed and r is the twist ratio (equal to 1 [1]).

The ONB point is evaluated using the Bergles-Rohsenow correlation:

$$h_{ST}(T_{ONB} - T_b) = 15500 \cdot p_b^{1.156} \cdot [1.8 \cdot (T_{ONB} - T_{sat}(p_b))]^n \quad (6)$$

where p_b is the coolant bulk pressure (MPa), T_{sat} is the coolant saturation temperature at its bulk pressure and the coefficient n is defined as follows:

$$n = \frac{2.046}{p_b^{0.0234}} \quad (7)$$

The heat transfer coefficient is then evaluated in partial boiling using the Bergles-Rohsenow method:

$$h = \frac{q_{tot}}{(T_w - T_b)} \quad (8)$$

where q_{tot} is the total heat flux transferred defined as:

$$q_{tot} = \sqrt{q_{ST}^2 + q_{nb}^2 \cdot \left(1 - \frac{q_0}{q_{nb}}\right)^2} \quad (9)$$

q_{ST} is the heat flux in single phase evaluated through the Sieder-Tate correlation:

$$q_{ST} = h_{ST} \cdot (T_w - T_b) \quad (10)$$

q_{nb} is the heat flux in subcooled boiling calculated through the Thoms-CEA correlation defined in Eq. (11) and q_0 is q_{nb} at $T_w = T_{ONB}$.

$$q_{nb} = 10^6 \cdot \left(\exp\left(\frac{p_b}{8.7}\right) \cdot \frac{(T_w - T_{sat})}{22.65} \right)^{2.8} \quad (11)$$

The implementation of a temperature-dependent heat transfer coefficient in FreeFEM++ is in principle feasible, however, since the model aims at being fast and giving a reasonable estimate of the thermal response of the monoblocks, the implementation of the heat transfer coefficient has been simplified. In fact, the correlations above make use of several thermal properties of the coolant, which should be implemented in FreeFEM++, and are nonlinear, which means that should be solved with an ad-hoc method.

Therefore, using the Matlab routine implementing the correlations above, the heat transfer coefficient has been evaluated for several wall and bulk temperatures at an average pressure of 3.65 MPa between the outer VT inlet and the inner VT outlet and a fluid speed of 8.4 m/s [3] producing the results in Figure 14.

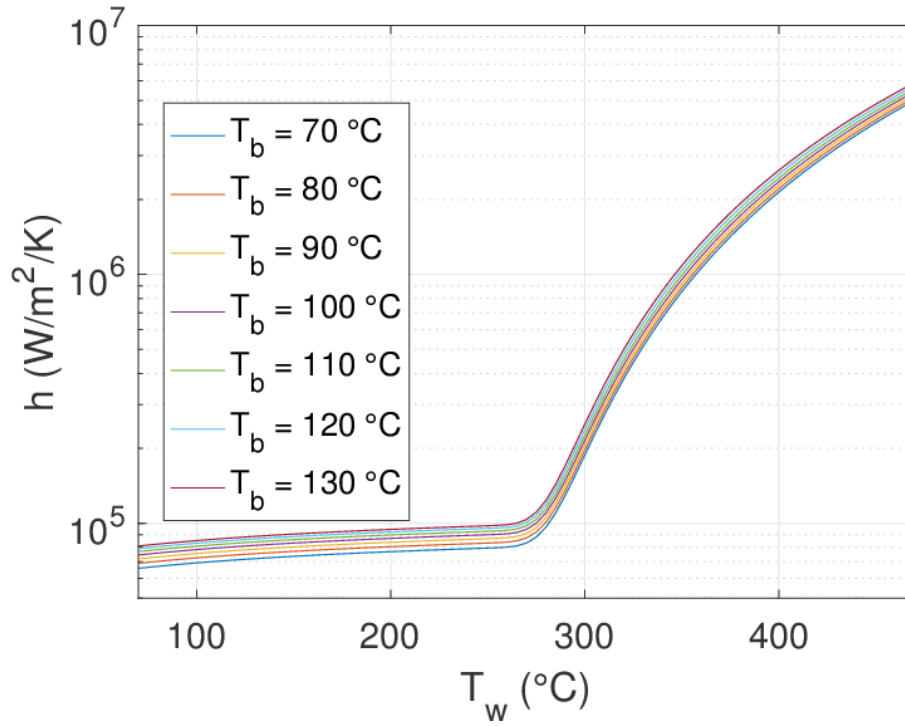


Figure 14: parametric plot of the heat transfer coefficient for the monoblocks.

At this point, the curve at $T_b = 90$ °C, which is within the range expected for the VTs cooling [3], has been chosen to be the basis of a polynomial fit of the heat transfer coefficient calculated with respect to the wall temperature T_w , which is the independent variable in the FreeFEM++ simplified model. The curves in Figure 14 clearly show two different behaviors for T_w larger or smaller than ~ 270 °C, therefore, the polynomial fit has been split in two parts, one valid up to 270 °C and one from 270 °C onwards. The following equations are the polynomial fits calculated and have been implemented in the simplified model:

$$h(T_w) = 2.1415 \times 10^{-3} \times T_w^3 - 1.2231 \times T_w^2 + 2.6941 \times 10^2 \times T_w + 5.6162 \times 10^4 \quad \text{for } T_w \leq 270 \text{ °C} \quad (12)$$

$$h(T_w) = -2.6549 \times 10^{-7} \times T_w^5 + 4.6179 \times 10^{-4} \times T_w^4 - 0.32055 \times T_w^3 + 2.4134 \times 10^2 \times T_w^2 - 8.9665 \times 10^4 \times T_w + 1.0945 \times 10^7 \quad \text{for } T_w > 270 \text{ °C} \quad (13)$$

Figure 15 and Figure 16 show the values of the heat transfer coefficient h for the temperature ranges adopted and the polynomial fits found. Of course, the

implemented equations are valid only for water at ~ 3.65 MPa, $T_b \sim 90$ °C and the geometrical data of the monoblock as reported previously in this document. However, in the simplified model an approximation is made and the polynomials are adopted also for coolant bulk temperature different from 90 °C, thus making a relative error on the actual heat transfer coefficient of $\sim 6.5\%$ in average and $\sim 18\%$ maximum.

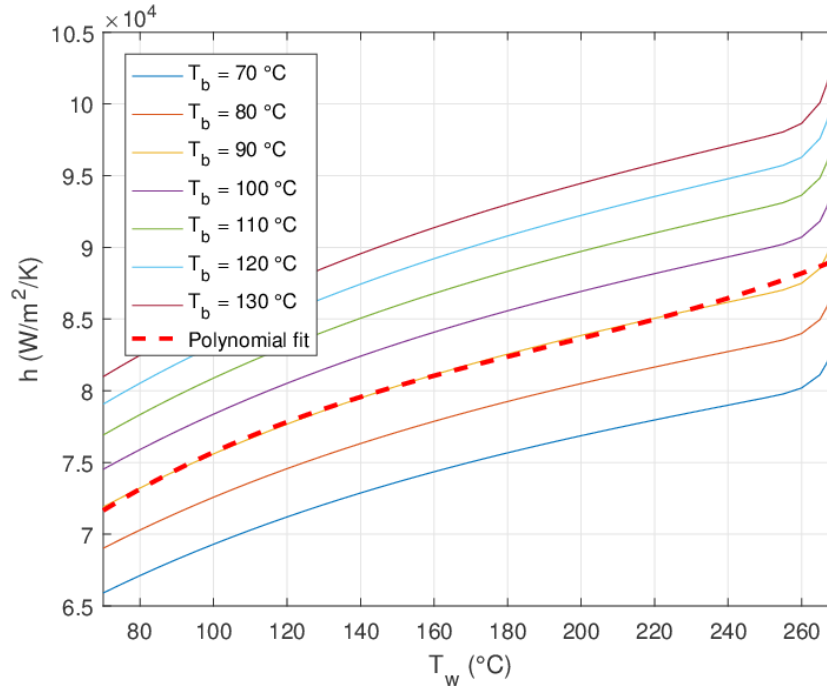


Figure 15: parametric plot of the heat transfer coefficient for the monoblocks with the polynomial fit on the lower temperature range.

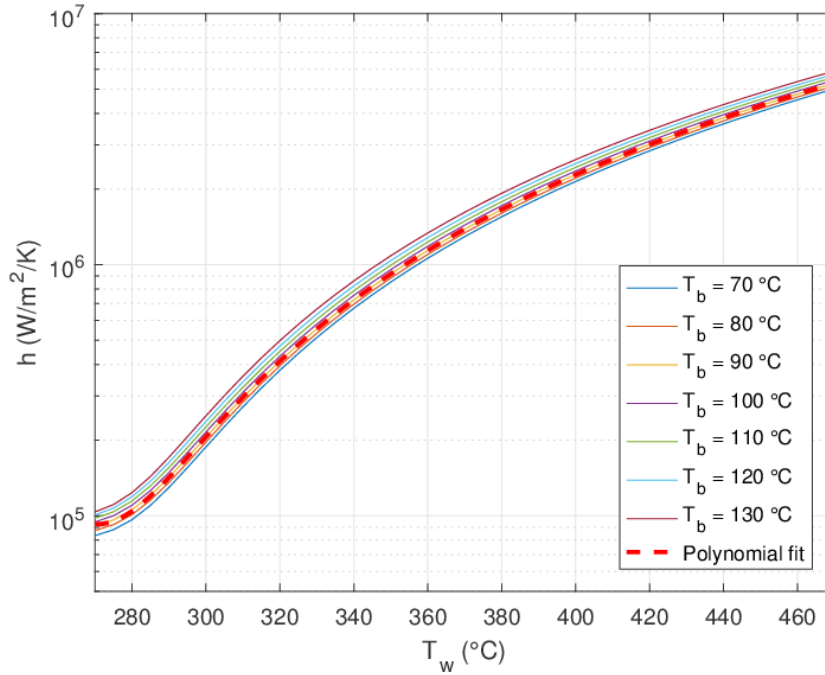


Figure 16: parametric plot of the heat transfer coefficient for the monoblocks with the polynomial fit on the upper temperature range.

Additionally, it must be noted that critical heat flux (CHF) conditions would likely appear for high wall temperature, dramatically reducing the heat transfer coefficient. This phenomenon is not modeled here and a limit of 40 MW/m² has been chosen as a threshold at which critical heat flux comes in and implemented within FreeFEM++ which will warn the user that this limit has been overcome without stopping the simulation or reducing the heat transfer coefficient.

To evaluate the wall critical heat flux (WCHF) the Tong75 correlation has been used, which provides the value of the critical heat flux through the following equation:

$$WCHF_{Tong75} = 0.23 \cdot f_0 \cdot \rho \cdot V \cdot i_{fg} \cdot \left[1 + 0.00216 \cdot \left(\frac{p_b}{p_0} \right)^{1.8} \cdot Re^{0.5} \cdot Ja \right] \quad (14)$$

in which the factor f_0 and Ja are defined in the equations below, V has been defined previously in Eq. (5), i_{fg} is the latent heat of vaporization and the pressure p_0 is equal to 22.09 MPa.

$$f_0 = 8 \frac{\left(\frac{D_h}{D_0}\right)^{0.32}}{Re^{0.6}} \quad (15)$$

where $D_0 = 12.7$ mm.

$$Ja = c_p \cdot \frac{T_{sat} - T_b}{i_{fg}} \cdot \frac{\rho_l(T_b)}{\rho_v(T_{sat})} \quad (16)$$

For the twisted tape configuration, the Tong75 correlation is modified with an experimental correction factor $C_f = 1.67$ [12]:

$$WCHF = C_f \cdot WCHF_{Tong75} \quad (17)$$

For the range of temperature foreseen for the heat transfer coefficient calculation, the calculated value of the WCHF is around or greater than 40 MW/m², well in line with the value reported in [11].

If the user needs/prefers different treatment of the heat transfer coefficient or the critical heat flux for its own particular application, the changes can be easily done in the FreeFEM++ scripts.

2.1.3 Benchmark

Given the monoblock heat transfer model described before, for which the FreeFEM++ script is available in Appendix 2, a benchmark has been carried out using the results of ANSYS calculations [13]. The comparison has been done using different values of heat flux on the top surface, not including chamfer, tilting and strike angle, and using as coolant temperature value of 100 °C. Moreover, the heat transfer coefficient has been simplified in both the ANSYS and FreeFEM++ models, assuming a constant value of 80 kW/m²/K for heat fluxes lower or equal than 11 MW/m² and a value of 200 kW/m²/K for heat fluxes larger than 20 MW/m². The results of the benchmark are reported in Figure 17 and in Table 2 in terms of temperature at the center and corner of the top surface, for two different values of the armor thickness, showing a good agreement between the two models. Also, in Figure 18 an example of the 2D temperature distributions obtained through the ANSYS and FreeFEM++ heat transfer models is reported. It can be seen that the 2D profiles match very well with each other, both qualitatively and quantitatively.

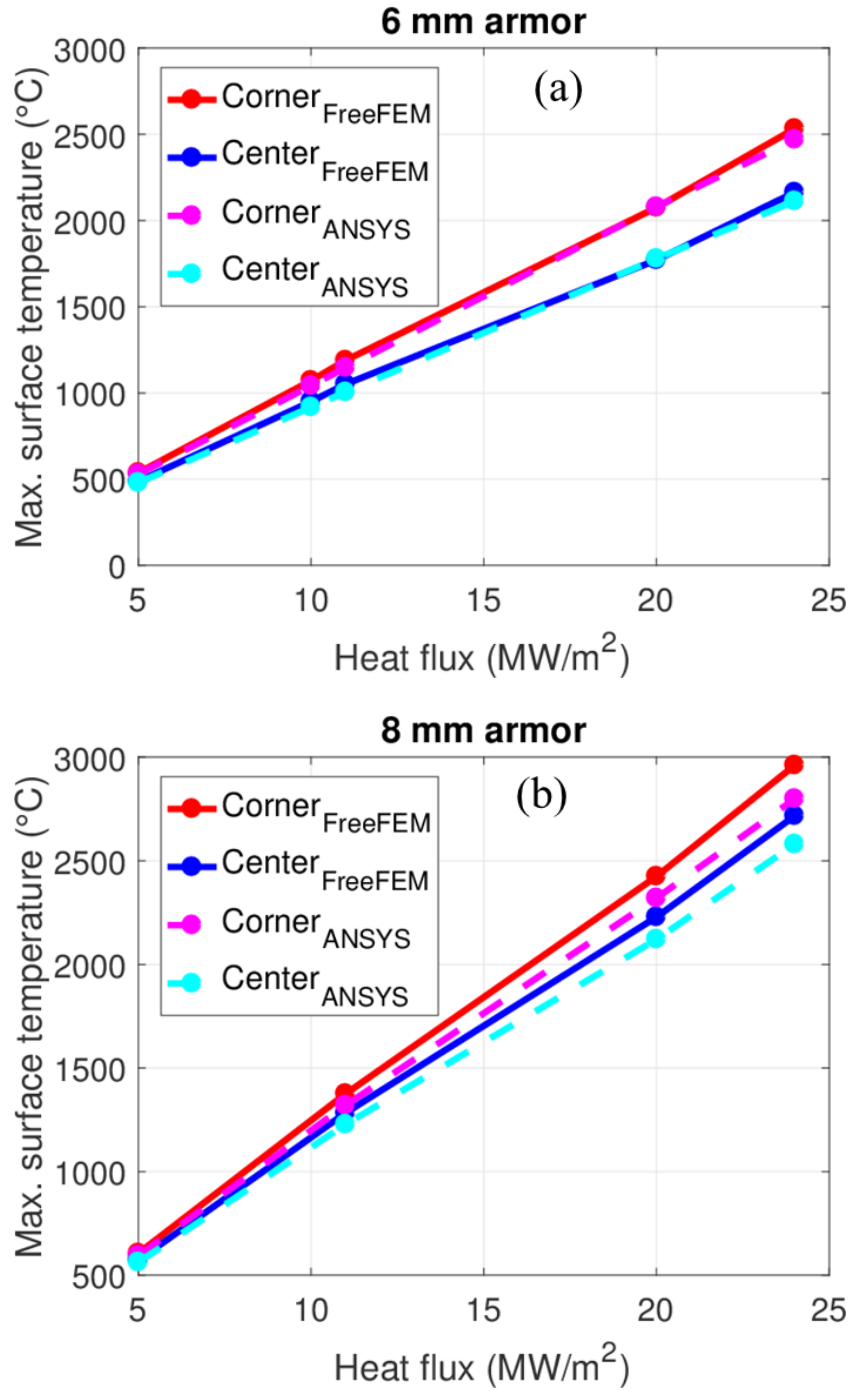
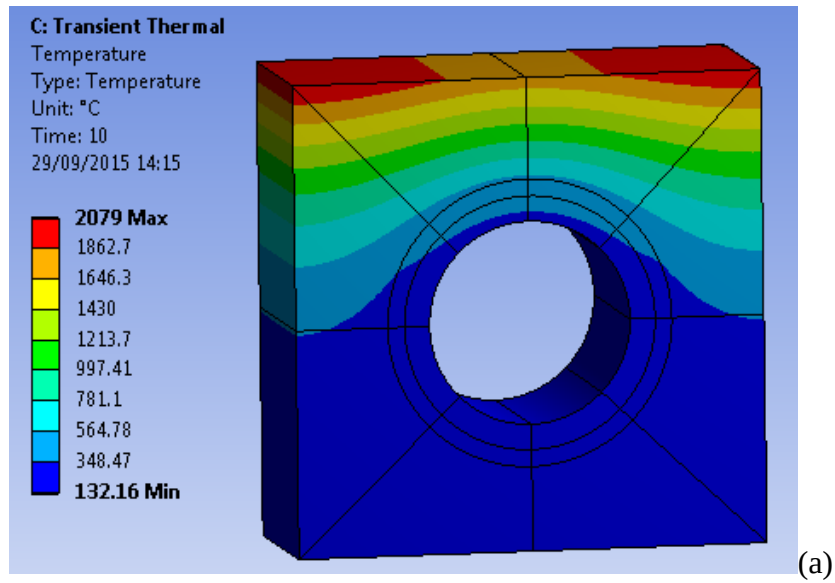


Figure 17: results of the benchmark between the FreeFEM++ heat transfer model and the ANSYS model for a W armor of 6 mm (a) and 8 mm (b). The solid lines in red (blue) refer to the FreeFEM++ values for the corner (center) of the top surface, while the dashed lines in magenta (cyan) refer to the ANSYS values for the corner (center) of the top surface.

Table 2: results of the benchmark between the FreeFEM++ heat transfer model and the ANSYS model.

W armor	q (MW/m²)	T_{corner} (°C)		T_{center} (°C)	
		FreeFEM++	ANSYS	FreeFEM++	ANSYS
6 mm	5	537	530	487	480
	10	1071	1042	950	918
	11	1189	1147	1051	1005
	20	2075	2079	1771	1779
	24	2532	2470	2163	2113
8 mm	5	605	595	572	563
	11	1375	1320	1283	1228
	20	2424	2320	2228	2121
	24	2961	2799	2716	2580



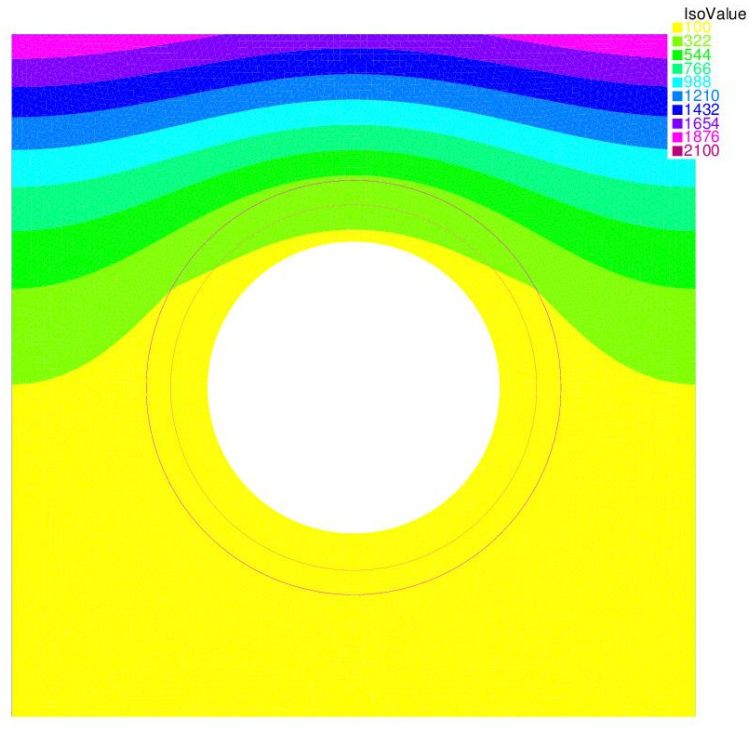


Figure 18: result of the benchmark between the ANSYS (a) and FreeFEM++ (b) heat transfer models for the case of W armor of 6 mm and a heat flux of 20 MW/m².

Another benchmark of the heat transfer model has been carried out at LECAD laboratory in Ljubljana [14] using Code Aster [15]. The only modeling difference with respect to the previous benchmark is that a Dirichlet boundary condition has been applied on the inner diameter of the pipe instead of a convective heat transfer condition. The fixed temperature imposed here is again 100 °C. Moreover, the study has focused only on a W armor thickness of 6 mm. The results of the benchmark, reported in Table 3 for what concerns the top surface center and corner temperatures, and in Table 4 in terms of maximum temperature of the Cu interlayer a and CuCrZr pipe, show a very good agreement between the two models.

Table 3: results of the benchmark between the FreeFEM++ heat transfer model and the Code Aster in terms of W top surface corner and centerline temperature.

q (MW/m ²)	T_{corner} (°C)		T_{center} (°C)	
	FreeFEM++	Code Aster	FreeFEM++	Code Aster
5	438	437	391	390
10	843	841	731	729
11	932	929	806	803
20	1809	1804	1544	1539
24	2215	2209	1894	1886

Table 4: results of the benchmark between the FreeFEM++ heat transfer model and the Code Aster model in terms of maximum Cu interlayer and CuCrZr pipe temperatures.

q (MW/m ²)	$T_{\text{max,Cu}}$ (°C)		$T_{\text{max,CuCrZr}}$ (°C)	
	FreeFEM++	Code Aster	FreeFEM++	Code Aster
5	153	156	134	137
10	207	213	168	175
11	217	225	175	183
20	314	328	236	250
24	357	374	263	279

2.2 Reflector plates

The same simplifying approach described before and adopted for the monoblock has been also applied for the hypervapotron channels of the RPs, see the cross section in Figure 3 and Figure 19. The dented structure of the CuCrZr has been reproduced also in the FreeFEM++ model, see Figure 19, where a Robin boundary condition, *i.e.* heat transfer with coolant, has been applied. The top surface, as in the case of the monoblock, experiences the plasma/neutrals heating and therefore the applied BC is an imposed heat flux. The outer edges are supposed to be adiabatic exactly as in the case of the monoblock.

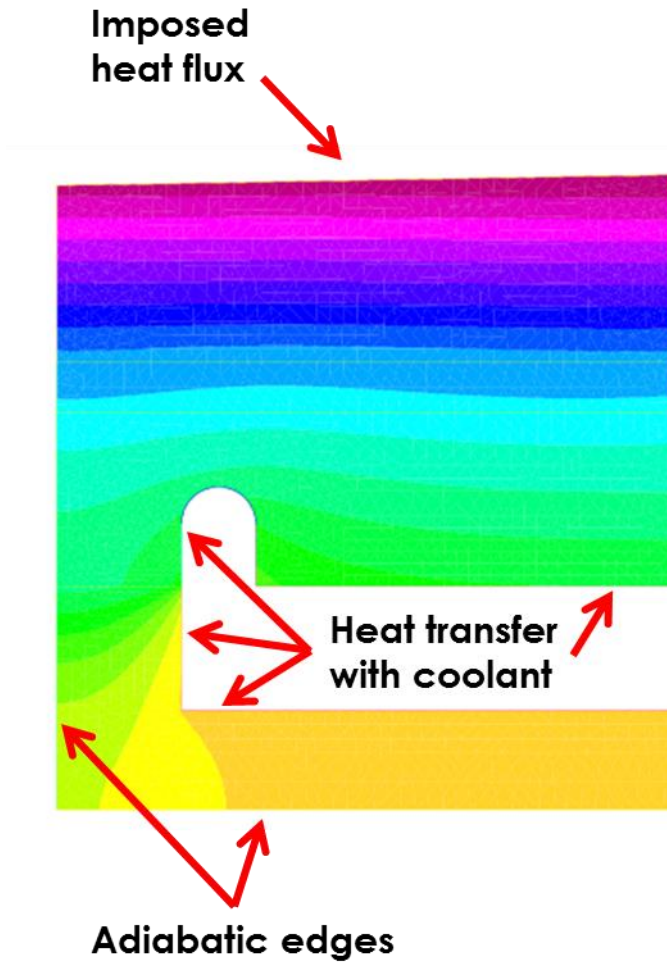


Figure 19: 2D cross section of the hypervapotron along poloidal direction with description of the boundary conditions applied.

2.2.1 Geometry

The geometrical data for the hypervapotron cross section along the poloidal direction are shown in Figure 21 and listed in Table 6 [13]. Note that each of the quantities reported can be modified by the user through the ad-hoc input file described in section 3.2.2.

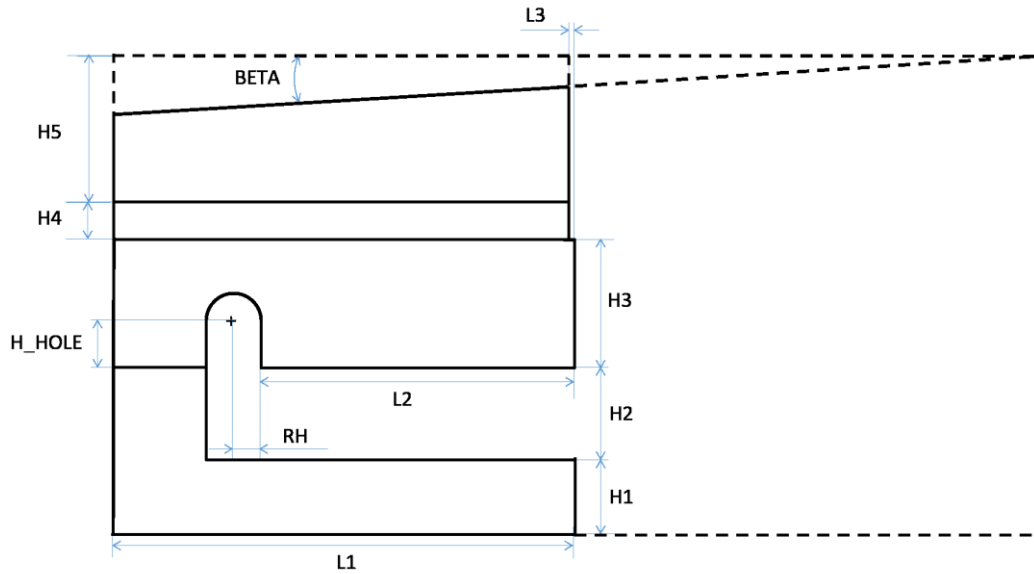


Figure 20: 2D cross section of the hypervapotron along poloidal direction with description of the geometrical quantities.

Table 5: geometrical data for the hypervapotron heat transfer model.

Quantity	Description	Value
L1	Half of the tile toroidal length	25 mm
L2	Half of the CuCrZr lower surface in contact with coolant (excluding dent)	17 mm
L3	Half of the gap in the castellated top structure	0.25
H1	Thickness of the SS base	4 mm
H2	Thickness of the coolant flow area (excluding dent)	5 mm
H3	Thickness of the CuCrZr section	7 mm
H4	Thickness of the Cu interlayer	2 mm
H5	Thickness of the W armor	8 mm
H_HOLE	Distance between CuCrZr lower surface and dent center	2.5 mm
RH	Radius of the dented semi-circumference	1.5 mm
P_LENGTH	Tile poloidal length / poloidal discretization step	20 mm
T_GAP	Toroidal gap between adjacent hypervapotrons	1 mm
ALPHA	Reflector plate tilting angle	1.0°
BETA	Toroidal chamfer angle	1.0°
THETA	Toroidal strike angle of the magnetic field lines	4.2°

2.2.2 Heat transfer coefficient

For the hypervapotron, the same correlations adopted for the monoblocks have been used. The only differences being in the hydraulic diameter to be used which is defined as:

$$D_h = 4 \cdot \frac{(W \cdot H - w \cdot h)}{W + w + 2 \cdot (H + h)} \quad (18)$$

for which the geometrical dimensions W , w , H and h are explained in Figure 21.

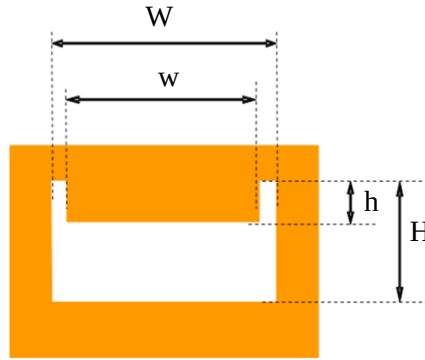


Figure 21: cross section of the hypervapotron cooling configuration.

Moreover, the fin effect factor in the Sieder-tate correlation, see Eq. (3), for the hypervapotron is $f = 1$.

Similarly as it has been done for the monoblocks, the Matlab routine has been used for calculating parametrically different values of the heat transfer coefficient. The pressure adopted is 2.95 MPa, an average value between the design pressure at the RPs inlet and outlet [3], and a speed of 5 m/s [3]. Figure 22 shows the results of the Matlab routine.

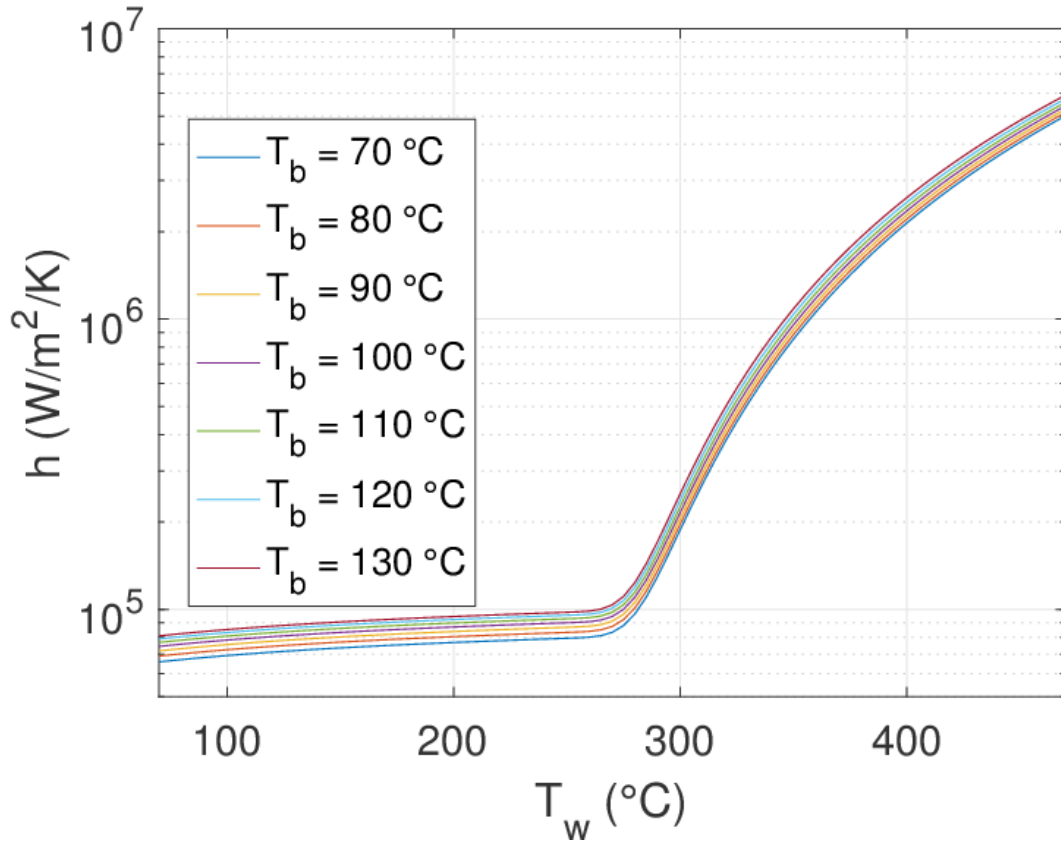


Figure 22: parametric plot of the heat transfer coefficient for the hypervapotron.

Similarly as for the monoblocks, a bulk temperature of 110 °C, within the expected operational range of the ITER RPs [3], has been chosen as the basis of the polynomial fit, which also in this case has been split into two expressions, one for wall temperatures lower than 250 °C and one for higher temperatures, see the following equations.

$$h(T_w) = 8.7094 \times 10^{-4} \times T_w^3 - 0.49289 \times T_w^2 + 1.2263 \times 10^3 \times T_w + 2.5198 \times 10^4 \quad \text{for } T_w \leq 250 \text{ } ^\circ\text{C} \quad (19)$$

$$h(T_w) = 1.912 \times 10^{-7} \times T_w^5 - 4.3537 \times 10^{-4} \times T_w^4 + 0.3693 \times T_w^3 + 40.9828 \times T_w^2 - 2.5576 \times 10^4 \times T_w + 4.7393 \times 10^6 \quad \text{for } T_w > 250 \text{ } ^\circ\text{C} \quad (20)$$

Figure 23 and Figure 24 show the values of the heat transfer coefficient h for the temperature ranges adopted and the polynomial fits found. The comments on the validity range of the monoblock's fits are still valid here as well and looking at

the relative error with respect to the actual heat transfer coefficient the average is ~7% and the maximum is ~24%.

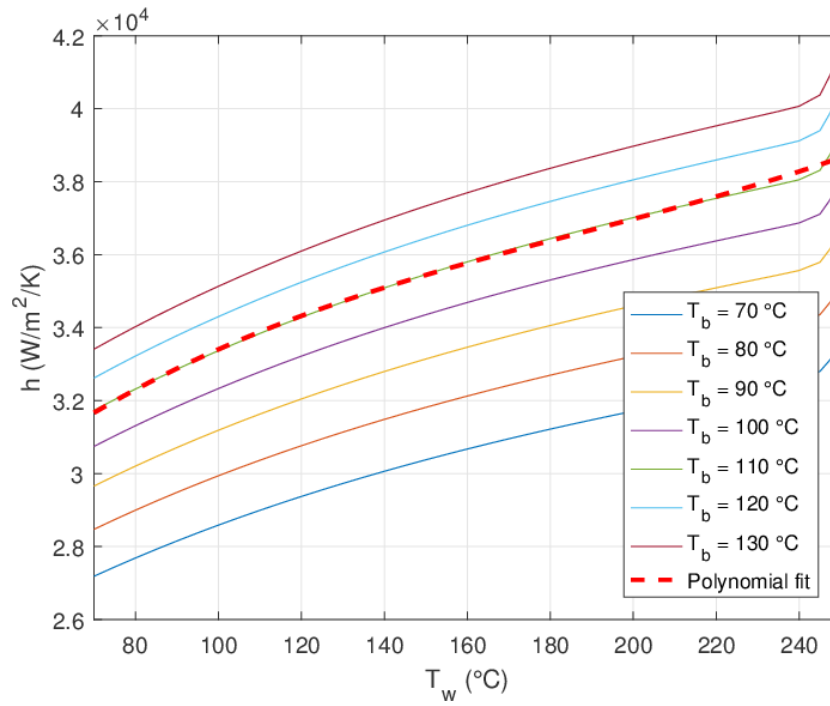


Figure 23: parametric plot of the heat transfer coefficient for the hypervapotron with the polynomial fit on the lower temperature range.

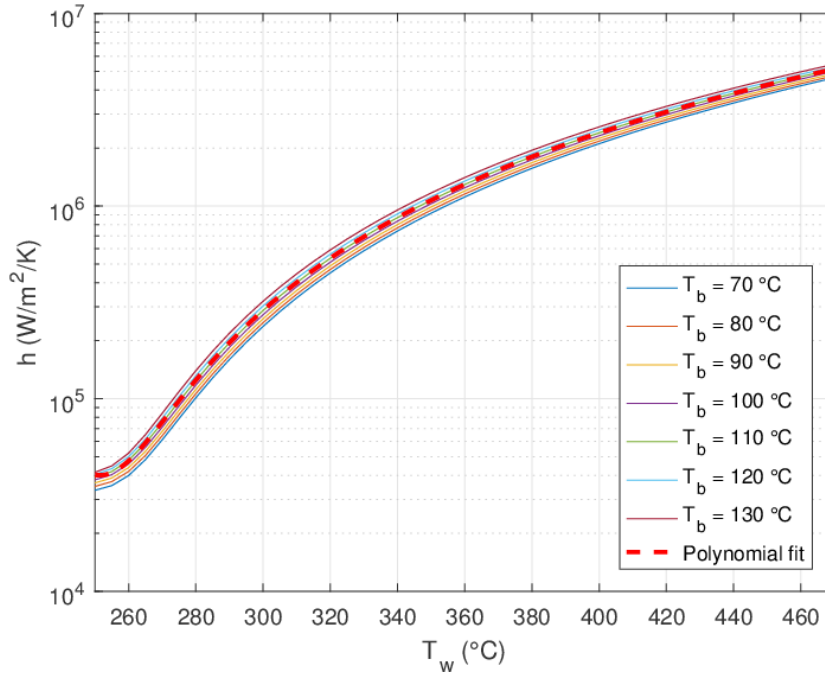


Figure 24: parametric plot of the heat transfer coefficient for the hypervapotron with the polynomial fit on the upper temperature range.

Concerning the critical heat flux threshold, the same Tong75 correlation adopted for the monoblock has been used here, being the only difference the correction factor C_f , which for the hypervapotron is defined as follows [16]:

$$C_f = 0.2584 \cdot W^{-0.5249} \quad (21)$$

where W is the hypervapotron channel width defined in in Figure 21.

For the temperature range adopted in the heat transfer coefficient calculation the Tong75 corrected correlation gives WCHF values around 20 MW/m² and higher. In order to be conservative, the lower value found, 20 MW/m², has been chosen as threshold after which the FreeFEM++ model would warn the user, without stopping the simulation.

Chapter 3

How to use it

3.1 Definition of plot zones in DG

In order to have the heat load profiles on the VTs and RPs, 4 different plot zones must be defined using DG, see SOLPS-ITER manual [10]. The steps to be followed are listed below.

1. Open your DG model (dg <name_of_your_model>.dg);
2. Add a plot zone: Variables → Add → Plot zone, see Figure 25;
3. Define the **List of elements**: mark the wall segments belonging to the inner VT and click on ‘Set’;
4. Define the **Starting element**: mark the first element in COUNTER CLOCKWISE DIRECTION within the *List of elements* defined before and click on ‘Set’;
5. Define the **Zone label**: type in the empty box the string ‘trgi’ (without quotes) and click on ‘Set’;
6. Repeat step 2 → 5 by adding a plot zone for the outer VT (using label ‘trgo’), inner RP (using label ‘irp’) and outer RP (using label ‘orp’);
7. Output the .dgo file: File → Output;

8. Make sure that the *param.dg* file in your baserun points to the new *.dgo* file.

In case you don't have the DG model you can modify the *param.dg* file in order to define the 4 plot zones described above.

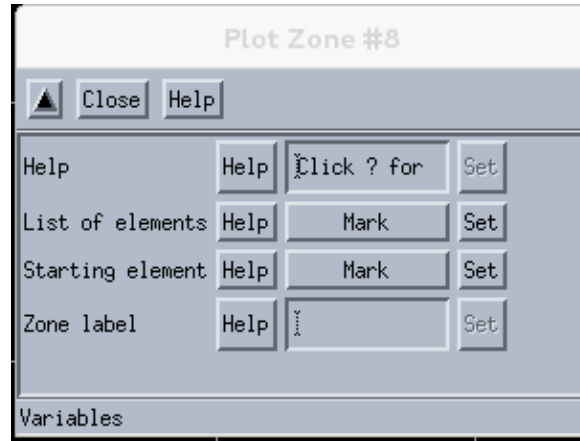


Figure 25: Picture of the Plot Zone definition mask in DG.

3.2 Formatted input files

Before starting the calculation, two additional formatted input files are needed: *b2ymb.dat* and *b2yrp.dat*. Both of them contain the geometrical data for the VT (the first) and the RP (the second), data for the coolant and data for the desired output files. A description of the input files and the format can be found in the next sections.

3.2.1 Monoblock input: b2ymb.dat

The input file needed by the monoblock heat transfer model, named *b2ymb.dat*, is a formatted file needed by an ad-hoc FORTRAN routine within SOLPS-ITER, *b2ymb.F*, see the manual [10], which reads it and then convert it into a file which is readable by the FreeFEM++ script. Four namelists must be present in the file, defined by the *ampersand* character “&”, one for the geometrical data, one for the mesh refinement, one for the coolant data and one for the output data. Every namelist has to be ended by the *backslash* character “\”. The comments after the value of the different parameters are initiated by the *exclamation mark* character “!”.

The geometrical data namelist contains the same quantities described in section 2.1.1 with the additional parameter *REVERSE*, which allows to flip / reverse the heat flux profile provided by SOLPS-ITER. This is needed for example for the inner VT, for which, as the plot zone must go in counter clockwise direction, the profile of the heat flux is given in an order opposite to the coolant flow, see Figure 7.

The only parameter present in the mesh namelist allows to control the refinement of the finite element mesh created by FreeFEM++: it controls the number of points on the longest edge (*i.e.* the three outer edges of the monoblock). The number of points on the three circumferences defined by the Cu interlayer and CuCrZr pipe are multiples of this parameter, therefore by increasing this integer number the whole mesh would be refined.

In the coolant namelist the user can control the value of the coolant temperature at the inlet of the monoblock, the mass flow rate flowing through the monoblock and the specific heat capacity of the coolant (default values are for water at operating conditions given in [3]).

Finally, the output namelist allows the user to control the output that the FreeFEM++ script allows. In particular, the first parameter *NPOINTS* defines the number of points on the top surface of the monoblock on which the user can check and print in a file (called *Tout_VT_[i-o].dat*) the temperature, while the second parameter *LPOINTS*, which has to be a vector of *NPOINTS* real numbers, defines the horizontal location (*i.e.* in the toroidal direction, with origin set in the bottom left corner) of the above mentioned points on which the user wants to check the temperature value. The last parameter of the namelist, *NPLOTS*, defines the interval at which a 2D temperature profile is saved, excluding the first and last discretized points along the poloidal direction, *i.e.* *NPLOTS* = 2 means that a 2D temperature profile is saved at points 1, 3, 5, 7, ..., N.

```

&GEOMETRY
HEIGHT          =      28.0e-3,      ! tile height [m]
T_LENGTH        =      28.0e-3,      ! tile toroidal length [m]
CU_TH           =      1.0e-3,      ! Cu interlayer thickness [m]
CUCRZR_TH       =      1.5e-3,      ! CuCrZr tube thickness [m]
DIAMETER        =      12.0e-3,      ! CuCrZr tube internal diameter [m]
W_TH            =      6.0e-3,      ! W armor thickness (from plasma facing
                                surface to Cu interlayer) [m]
P_LENGTH        =      1.2e-2,      ! tile poloidal length [m]
T_GAP           =      0.5e-3,      ! Toroidal gap between monoblocks [m]
BETA            =      1.0,          ! poloidal chamfer [degrees]
ALPHA           =      0.5,          ! Vertical target tilting [degrees]
THETA           =      4.2,          ! Toroidal strike angle of magnetic flux
                                lines [degrees]
REVERSE         =      0,            ! Integer, if >0 reverse the heat flux
                                profile from top to bottom
/
&MESH
NN              =      50,           ! # of mesh points on longest edge, can be
                                used to refine the mesh
/
&COOLANT
T_COOLANT       =      70.0,         ! Coolant inlet temperature [C]
M_FLOW          =      0.734,        ! Coolant mass flow rate [kg/s]
CP_COOLANT      =      4.22e3,       ! Coolant specific heat capacity [J/kg/K]
/
&OUTPUT
NPOINTS         =      2,            ! Number of points for output surface
                                temperature
LPOINTS         =      0., 14.0e-3,  ! horizontal location of points on the top
                                surface in absolute value [m] where the
                                temperature is saved in the file
NPLOTS          =      10,           ! Plot the 2D profile each NPLOTS cuts
/

```

3.2.2 Reflector plate input: b2yrp.dat

The input file needed by the reflector plate heat transfer model, named *b2yrp.dat*, is a formatted file basically identical to its monoblock counterpart. The ad-hoc FORTRAN routine within SOLPS-ITER, *b2yrp.F*, see the manual [10], reads this file and then convert it into a file which is readable by the FreeFEM++ script. Again, four namelists must be present in the file, defined by the *ampersand* character “&”,

one for the geometrical data, one for the mesh refinement, one for the coolant data and one for the output data. Every namelist has to be ended by the *backslash* character “\”. The comments after the value of the different parameters are initiated by the *exclamation mark* character “!”.

The geometrical data namelist contains the same quantities described in section 2.2.1 with the additional parameter *REVERSE*, which, as explained before, allows to flip / reverse the heat flux profile provided by SOLPS-ITER, if needed.

The mesh refinement parameter *NN* controls in the case of the reflector plates the number of points on H4, which is the thickness of the Cu interlayer. The number of points on the other edges are proportional to this number, thus allowing to increase or decrease the mesh refinement.

Finally, the coolant and output namelist have the same kind of parameters as for the monoblock input file, see previous section.

&GEOMETRY

L1	=	25.0e-3,	! half of the tile toroidal length [m]
L2	=	17.0e-3,	! half of the CuCrZr lower surface in contact with coolant (excluding dent) [m]
L3	=	0.25e-3,	! half of the gap in the castellated top structure [m]
H1	=	4.0e-3,	! thickness of the SS base [m]
H2	=	5.0e-3,	! thickness of the coolant flow area (excluding dent) [m]
H3	=	7.0e-3,	! thickness of the CuCrZr section [m]
H4	=	2.0e-3	! thickness of the Cu interlayer [m]
H5	=	8.0e-3	! thickness of the W armor [m]
H_HOLE	=	2.5e-3	! distance between CuCrZr lower surface and dent center [m]
RH	=	1.5e-3	! radius of the dented semi-circumference [m]
P_LENGTH	=	1.2e-2,	! tile poloidal length [m]
T_GAP	=	1.0e-3,	! Toroidal gap between hypervapotron channels [m]
BETA	=	1.0,	! poloidal chamfer [degrees]
ALPHA	=	0.5,	! Reflector plate tilting [degrees]
THETA	=	4.2,	! Toroidal strike angle of magnetic flux lines [degrees]
REVERSE	=	0,	! Integer, if >0 reverse the heat flux profile from top to bottom

/

&MESH

NN	=	20,	! \# of mesh points on H4. can be used to refine the mesh
----	---	-----	---

/

&COOLANT

T_COOLANT	=	70.0,	! Coolant inlet temperature [C]
M_FLOW	=	0.734,	! Coolant mass flow rate [kg/s]
CP_COOLANT	=	4.22e3,	! Coolant specific heat capacity [J/kg/K]

/

&OUTPUT

NPOINTS	=	2,	! Number of points for output surface temperature
LPOINTS	=	0., 14.0e-3,	! horizontal location of points on the top surface in absolute value [m] where the temperature is saved in the file
NPLOTS	=	10,	! Plot the 2D profile each NPLOTS cuts

/

3.2 Executing the thermal analysis

Once the two input files are present, the user just needs to execute the script *ITER_divertor_HTM* in the run directory, which does the following actions that can be executed step by step by the user, if necessary:

1. Checks the presence of the files with the heat load profiles *wlld_trgi.dat*, *wlld_trgo.dat*, *wlld_irp.dat*, *wlld_orp.dat* and the related wall layout files *wlly_*.dat*. If one or more are not present it runs *b2plot* to create the files and copy them in the run directory.

```
$ echo '1000 wlld' | b2plot
```

```
$ cp ./b2pl.exe.dir/wlld_* .
```

2. Creates an input file for the outer VT, readable by the FreeFem++ script, calling the post processing routine *b2ymb* which needs the *b2ymb.dat* file to be present. If the user wants to modify some parameters for the outer VT calculation, it must be done before running *b2ymb*.

```
$ echo 'o' | b2plot b2ymb
```

3. Runs the FreeFem++ script.

```
$ FreeFem++ $SOLPSTOP/scripts/Monoblock_HTM.edp
```

4. When the FreeFem++ simulation is completed the user has to close the window with the 2D temperature distribution to continue to the next step.
5. Runs *b2ymb* a second time to produce the input file of the inner VT readable by FreeFem++. The outlet temperature from the outer VT is automatically used as inlet temperature for the inner VT. Moreover, since the plot zone has been defined in counter clockwise direction the heat load profile is automatically rearranged to follow the coolant flow path. If the user wants to modify some parameters for the inner VT calculation, it must be done before running *b2ymb* the second time.

```
$ echo 'i' | b2plot b2ymb
```

6. Runs the FreeFem++ script.

```
$ FreeFem++ $SOLPSTOP/scripts/Monoblock_HTM.edp
```

7. When the FreeFem++ simulation is completed the user has to close the window with the 2D temperature distribution to continue to the next step.
8. Creates an input file for the outer RP, readable by the FreeFem++ script, calling the post processing routine *b2yrp* which needs the *b2yrp.dat* file to be present. The outlet temperature from the inner VT is automatically used as inlet temperature for the inner RP. If the user wants to modify some parameters for the outer RP calculation, it must be done before running *b2yrp*.

```
$ echo 'i' | b2plot b2yrp
```

9. Runs the FreeFem++ script.

```
$ FreeFem++ $SOLPSTOP/scripts/ReflectorPlate_HTM.edp
```

10. Runs *b2yrp* a second time to produce the input file of the outer RP readable by FreeFem++. The outlet temperature from the inner RP is automatically used as inlet temperature for the outer RP. If the user wants to modify some parameters for the inner RP calculation, it must be done before running *b2yrp* the second time.

```
$ echo 'o' | b2plot b2yrp
```

11. Runs the FreeFem++ script

```
$ FreeFem++ $SOLPSTOP/scripts/ReflectorPlate_HTM.edp
```

After the fourth FreeFem++ simulation all the calculations are completed.

3.3 Standard output

As a standard output, all the FreeFem++ scripts will produce (units are °C and m):

- Plots of the 2D temperature distribution on the VT and RP cross section at the inlet, outlet and every 10 cuts with label *Temp2D_[VT-RP]_[i-o]_[j].eps* (*j* is the index of the cut).

- Plots of the 2D temperature distribution on the VT and RP cross section for the cut which has the maximum surface temperature with label *Temp2D_[VT-RP]_[i-o]_MAX.eps*.
- A series of *.dat* files containing the maximum temperature in Cu, CuCrZr, W and SS regions and the coolant temperature on each cut. The labels are: *T_[CCZ-Cu-W-SS-water]_[VT-RP]_[i-o].dat*. Files called *X_[tg-rp][i-o].dat* store on the first column the poloidal coordinate of each cut along the wall (the origin is the first point of the first element of the plot zone); on the 2nd and 3rd columns the *r* and *z* absolute coordinates of each cut.
- A series of *.dat* files containing the W surface temperature at the desired location specified in *b2ymb.dat* or *b2yrp.dat*. Label: *Tout_[VT-RP]_[i-o].dat*

3.4 Windows/iOS users

For users that do not have FreeFEM++ installed on the same machine with which SOLPS-ITER is run, they only need to install FreeFEM++ on their operating system, see the installation page on the website <http://www.freefem.org/>. Then, they have to make sure that the file *b2ymb.dat*, *b2yrp.dat*, *wll[d-y]_trg[o-i].dat*, *wll[d-y]_[i-o]rp.dat*, *b2ymb* and *b2yrp* executables, *Monoblock.edp* and *ReflectorPlate.edp* are present in the directory where you want to perform the calculation (the FreeFEM++ scripts in *.edp* format that are available in *\$SOLPSTOP/scripts* can be run indistinctively on every operating system, while the executable files for *b2ymb* and *b2yrp* can be compiled from the sourcefiles *b2ymb.F* and *b2yrp.F* in *\$SOLPSTOP/modules/B2.5/src/postprocessing*. When these basic requirements are satisfied, the users should:

1. Edit the file *b2ymb.dat* with the desired values. (The flag *REVERSE* must be set to 0 for ITER cases);
2. Run *b2ymb.exe*. When asked, type 'o' (without quotes) and *RETURN*;
3. Run the FreeFem++ script *Monoblock_HTM.edp*;
4. When the FreeFem++ simulation is completed, edit the file *b2ymb.dat* and update the coolant inlet temperature with the outlet one from the outer VT (it is written on the command window of the FreeFem++ executable and in

the last row of the file *T_water_VT_o.dat*). (Update the flag *REVERSE=0* in *REVERSE=1* for ITER cases);

5. Run *b2ymb.exe*. When asked, type 'i' (without quotes) and *RETURN*;
6. Run the FreeFem++ script *Monoblock_HTM.edp*;
7. When the FreeFem++ simulation is completed, edit the file *b2yrp.dat* and update the coolant inlet temperature with the outlet one from the inner VT (it is written on the command window of the FreeFem++ executable and in the last row of the file *T_water_VT_i.dat*). (The flag *REVERSE* must be set to 0 for ITER cases);
8. Run *b2yrp.exe*. When asked, type 'i' (without quotes) and *RETURN*;
9. Run the FreeFem++ script *ReflectorPlate_HTM.edp*;
10. When the FreeFem++ simulation is completed, edit the file *b2yrp.dat* and update the coolant inlet temperature with the outlet one from the inner RP (it is written on the command window of the FreeFem++ executable and in the last row of the file *T_water_RP_i.dat*). (The flag *REVERSE* must be set to 0 for ITER cases);
11. Run *b2yrp.exe*. When asked, type 'o' (without quotes) and *RETURN*;
12. Run the FreeFem++ script *ReflectorPlate_HTM.edp*;

The output files produced are the same as in the case of Linux systems.

References

- [1] R. Pitts, A. Kukushkin, A. Loarte, A. Martin, M. Merola, C. Kessel, V. Komarov and M. Shimada, "Status and physics basis of the ITER divertor," *Physics Scripta*, vol. T138, p. 014001, 2009.
- [2] R. Pitts, *Physics conclusions in support of ITER W divertor monoblock shaping*, Rome: 22nd PSI Conference, 2016.
- [3] M. Garcia-Martinez and S. Panayotis, "Divertor Input for ACP," ITER report RRNTQH, 2015.
- [4] R.A. Pitts, *private communication*, March 2017.
- [5] R. Pitts, S. Carpentier, F. Escourbiac, T. Hirai, V. Komarov, A. Kukushkin, S. Lisgo, A. Loarte, M. Merola, R. Mitteau, A. Raffray, M. Shimada and P. Stangeby, "Physics basis and design of the ITER plasma-facing components," *Journal of Nuclear Materials*, vol. 415, p. S957–S964, 2011.
- [6] M. Merola, F. Escourbiac, R. Raffray, P. Chappuis, T. Hirai and A. Martin, "Overview and status of ITER internal components," *Fusion Engineering and Design*, vol. 89, p. 890–895, 2014.
- [7] C. Baxi, "Thermal hydraulics of water cooled divertors," *Fusion Engineering and Design*, vol. 56–57, p. 195–198, 2001.
- [8] J. Milnes, A. Burns and D. Drikakis, "Computational modelling of the HyperVapotron cooling technique," *Fusion Engineering and Design*, vol. 87, p. 1647–1661, 2012.
- [9] F. Hecht, "New development in FreeFem++," *Journal of Numerical Mathematics*, vol. 20, pp. 251-265, 2012.
- [10] [Online]. Available:
https://portal.iter.org/departments/POP/CM/IMAS/SOLPS-ITER/Manuals%20and%20Documentation/SOLPS-ITER_User_Manual.pdf. [Accessed 14 January 2017].
- [11] V. Komarov, "Thermal analysis of ITER Divertor PFU W-monoblocks with local shaping at surface heat flux," ITER report KRTXGK, 2013.

- [12] G. Broyde and F. Escourbiac, "Development of a MATLAB routine for the calculation of the Heat Transfer Coefficient to be used for ITER Plasma Facing Components thermo-mechanical analyses," ITER_D_35L29Y, 2010.
- [13] S. Panayotis, *private communication*, June 2016..
- [14] M. Brank, *ITER divertor monoblock FEM benchmark*, Ljubljana: LECAD Laboratory, University of Ljubljana, 2017.
- [15] [Online]. Available: <http://www.code-aster.org/spip.php?rubrique2>. [Accessed 14 June 2017].
- [16] F. Escourbiac, J. Schlosser, M. Merola and I. Bobin Vastra, "Experimental optimisation of a hypervapotron concept for ITER plasma facing components," *Fusion Engineering and Design*, Vols. 66-68, pp. 301-304, 2003.
- [17] "\\iter\cfs\public\panayos\HV," [Online].

Appendix 1

Material properties

Table 6: Material properties of CuCrZr [11].

Density (kg/m ³)	Specific heat (J/Kg/K)	Thermal conductivity (W/m/K)	Temperature (°C)
8900	390	318	20
8886	393	324	50
8863	398	333	100
8840	402	339	150
8816	407	343	200
8791	412	345	250
8767	417	346	300
8742	422	347	350
8716	427	347	400
8691	432	346	450
8665	437	346	500

8638	442	347	550
8611	447	349	600

The CuCrZr thermal conductivity has been fitted with a polynomial approximation as follows (temperature in °C):

$$k(T) = 2.578 \times 10^{-7} T^3 - 6.4511 \times 10^{-4} T^2 + 0.25768 T + 312.97 \quad (W/m/K) \quad (22)$$

Table 7: Material properties of Cu [11].

Density (kg/m ³)	Specific heat (J/Kg/K)	Thermal conductivity (W/m/K)	Temperature (°C)
8940	388	401	20
8926	-	-	50
8903	394	395	100
8879	-	-	150
8854	401	388	200
8829	-	-	250
8802	410	381	300
8774	-	-	350
8744	-	374	400
8680	430	367	500
8610	441	360	600
-	452	355	700

The Cu thermal conductivity has been fitted with a polynomial approximation as follows (temperature in °C):

$$k(T) = 3.0851 \times 10^{-8} T^3 - 2.2771 \times 10^{-5} T^2 - 6.6865 \times 10^{-2} T + 402.16 \quad (W/m/K) \quad (23)$$

Table 8: Material properties of W [11].

Density (kg/m ³)	Specific heat (J/Kg/K)	Thermal conductivity (W/m/K)	Temperature (°C)
1930	129	173	20
-	130	-	50
1928	132	165	100
1925	135	156	200
1923	138	147	300
1920	141	140	400
1918	144	133	500
1915	147	127	600
1913	150	122	700
1910	152	118	800
1907	155	114	900
1904	158	110	1000
1901	160	-	1100
1899	163	105	1200
-	165	-	1300
1892	168	102	1400

1889	170	101	1500
-	172	-	1600
1880	176	-	1800
1875	180	99.1	2000
1860	189	98.7	2500
-	190	-	2600
-	192	96	2800
1847	-	-	3000
1840	194	90	3200

The W thermal conductivity has been fitted with a polynomial approximation as follows (temperature in °C):

$$\begin{aligned}
 k(T) = & 2.7636 \times 10^{-16} T^5 - 2.0204 \times 10^{-12} T^4 - 2.7802 \times 10^{-9} T^3 \\
 & + 4.5169 \times 10^{-5} T^2 - 0.10545 T \\
 & + 175.07 \quad (W/m/K)
 \end{aligned} \tag{24}$$

Table 9: Material properties of 316L SS [17].

Density (kg/m ³)	Temperature (°C)
7930	50
7899	100
7879	150
7858	200
7837	250

7815	300
7793	350
7770	400
7747	450
7724	500
7701	550
7677	600
7654	650
7630	700

Thermal conductivity of 316L SS (temperature in °C) [17]:

$$k(T) = 1.502 \times 10^{-2} T + 13.98 \quad (W/m/K) \quad (25)$$

Specific heat of 316L SS (temperature in °C):

$$k(T) = 462.69 + 0.520265T - 1.7117 \times 10^{-3} T^2 + 3.3658 \times 10^{-6} T^3 - 2.1958 \times 10^{-9} T^4 \quad (J/kg/K) \quad (26)$$

Appendix 2

Script of the monoblock heat transfer model

```
//Load input data and define geometry
ifstream inputs("mnb_in.dat");
real
L,D1,D2,D3,Hw,Ltile,Tf,mflow,cpw,L1,aalpha,bbeta,ttheta;
real sss1,sss2,H,alpha,beta,theta,Tgap,Xshadow;
int n,npointout;
int inout,nplots;
string sinout;
inputs>>L1;      // tile height
inputs>>L;        // tile toroidal length
inputs>>sss1;     // Cu interlayer thickness
inputs>>sss2;     // CuCrZr tube thickness
inputs>>D3;       // CuCrZr tube internal diameter
inputs>>Hw;       // W armour thickness (from plasma facing
surface to Cu interlayer)
inputs>>Ltile;    // Tile poloidal length
inputs>>Tgap;     // Toroidal gap between monoblocks [m]
inputs>>alpha;    // VT tilting
inputs>>bbeta;    // Poloidal chamfer
inputs>>ttheta;    // Toroidal strike angle of magnetic
flux lines [degrees]
inputs>>Tf;       // Coolant inlet temperature
inputs>>mflow;    // Coolant mass flow rate
```



```
inputs>>cpw;      // Coolant specific heat
inputs>>n;         // # of mesh points on longest edge
inputs>>npointout; // Number of points for output
surface temperature
inputs>>inout; //defines inner (1) or outer (0)
inputs>>nplots; //2D plots are produce each nplots cuts
int iplot=0;
if (inout==1){
    sinout="i";}
else{
    if (inout==0){
        sinout="o";}
    else{
        sinout="unknown";}
}

real[int] Lout(npointout);
for (int kk=1;kk<=npointout;kk+=1){
    inputs>>Lout(kk-1);
}
D2=D3+2.0*sss2;
D1=D2+2.0*sss1;
H=L1-D1/2.0-Hw;

alpha=aalpha*pi/180.0;
beta=bbeta*pi/180.0;
theta=ttheta*pi/180.0;
Xshadow=(L*tan(beta)-
Tgap*tan(theta))/(tan(beta)+tan(theta));
//MATERIALS DATA
//Thermal conduct CuCrZr
real[int] CCZ(4);
CCZ(0)=5.257798071745529e-07;
CCZ(1)=-6.451101399520426e-04;
CCZ(2)=2.576784427085231e-01;
CCZ(3)=3.129694443435334e+02;
//Thermal cond Cu
real[int] CU(4);
CU(0)=3.085132448660574e-08;
CU(1)=-2.277103401239594e-05;
CU(2)=-6.686469397617298e-02;
```

```

CU(3)=4.021574134794590e+02;
//Thermal cond W
real[int] WW(6);
WW(0)=2.763634292770776e-16;
WW(1)=-2.020389882180738e-12;
WW(2)=-2.780235961230648e-09;
WW(3)=4.516924060319057e-05;
WW(4)=-1.054501953718856e-01;
WW(5)=1.750724560926432e+02;

//Load heat flux profile
ifstream size1("tg"+sinout+"_size.dat");
ifstream www1("Wtot_tg"+sinout+".dat");
ifstream www2("Wpls_tg"+sinout+".dat");
int Ntile;
size1>>Ntile;
real[int] qtot(Ntile);
real[int] qpls(Ntile);
for(int nn=0;nn<Ntile;nn+=1){
    www1>>qtot(nn);
    www2>>qpls(nn);
}
real[int] Tmax(Ntile);
real[int] TmaxCU(Ntile);
real[int] TmaxCCZ(Ntile);
real[int] TmaxW(Ntile);
real[int] Twater(Ntile);
real[int] Qreal(Ntile);
real[int] HTC(Ntile);
real[int,int] Tout(Ntile,npointout);
real TTMAX=0.0;
int IMAX=0;

//Domain generation
border A(t=0.,L){x=t;y=0.;;};
border B(t=0.,L1){x=L;y=t.;;};
border C(t=L,0.){x=t;y=L1+(t-L)*tan(beta).;;};
border D(t=L1-L*tan(beta),0.){x=0.0;y=t.;;};
border
EE(t=0.,2.*pi){x=L/2.+D1/2.*cos(t);y=H+D1/2.*sin(t).;;};
border
FF(t=0.,2.*pi){x=L/2.+D2/2.*cos(t);y=H+D2/2.*sin(t).;;};

```

```
border
GG(t=0.,2.*pi){x=L/2.+D3/2.*cos(t);y=H+D3/2.*sin(t);};
mesh
block=buildmesh(A(n)+B(n)+C(n)+D(n)+EE(3*n)+FF(2*n)+GG(-
2*n));
plot(block,fill=1,ps="Domain_V.eps");

//PROBLEM DEFINITION
fespace Vh(block,P1);
Vh T=Tf,v,Told,TTT;
int ww=block((L-D1)/2.-1e-4,H).region,cu=block((L-D2)/2.-
(D1-D2)/4.,H).region,ccz=block((L-D3)/2.- (D2-
D3)/4.,H).region;
Vh kCCZ,kCu,kW,K,htc,HF,qq,TMAX;

problem
divert(T,v)=int2d(block)(K*(dx(T)*dx(v)+dy(T)*dy(v)))-
int1d(block,C)(qq*v)
-
int1d(block,GG)(htc*Tf*v)+int1d(block,GG)(htc*T*v);
real gamma,npoints=300.0,hfmax,hflux,CHF=40.0e6;

//Loop on the monoblock length
//ofstream ff00("Qwater.dat");
//ofstream ff01("Qplasma.dat");
ofstream ff02("warning_VT_"+sinout+".log");
real flag=0.0;
TTMAX=0.0;
for(int jj=0;jj<Ntile;jj+=1){
    qq=(qpls(jj)/cos(beta+alpha))*(x>=Xshadow)+(qtot(jj)-
qpls(jj));
    Qreal(jj)=qpls(jj)/cos(beta+alpha)+qtot(jj)-qpls(jj);
    real err=1.0,tol=1.e-3;
    Told=Tf;
    real qwater=0.0,qplasma=0.0;
    //Solve pseudo transient on the single tile

    while(err>tol){
        kCCZ=CCZ(0)*T^3.0+CCZ(1)*T^2.0+CCZ(2)*T+CCZ(3);
        kCu=CU(0)*T^3.0+CU(1)*T^2.0+CU(2)*T+CU(3);
```

```

kW=WW(0)*T^5.0+WW(1)*T^4.0+WW(2)*T^3.0+WW(3)*T^2.0+WW(
4)*T+WW(5);

K=kW*(region==ww)+kCu*(region==cu)+kCCZ*(region==ccz);
htc=(2.1415e-3*(T^3.0)-
1.2231*(T^2.0)+2.6941e2*T+5.6162e+04)*(T<270.0)+(-2.6549e-
07*(T^5.0)+4.6179e-04*(T^4.0)-
0.32055*(T^3.0)+2.4134e2*(T^2.0)-
8.9665e+04*T+1.0945e+07)*(T>=270.0);
divert;
err=sqrt(int2d(block)((T-
Told)^2.0))/sqrt(int2d(block)(T^2.0));
Told=T;

} //end pseudo transient

Twater(jj)=Tf;
HTC(jj)=int1d(block,GG)(htc)/int1d(block,GG)(1.0);

//Evaluate next T bulk
qwater=(-
int1d(block,GG)(htc*Tf)+int1d(block,GG)(htc*T))*2.0*Ltile;
Tf=Tf+qwater/mflow/cpw;

//Check for CHF (40 MW/m^2)
HF=htc*(T-Tf);
hfmax=0.0;
for (int ii=0;ii<=npoints;ii+=1.0){
    gamma=ii/npoints*pi/2.0;
    hflux=HF(L/2.+D3/2.*cos(pi/2.-
gamma),H+D3/2.*sin(pi/2.-gamma));
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
if(hfmax>=CHF){
    ff02 << "Warning! Reached critical heat flux of
"<<CHF<<" W/m^2. Tile number " <<jj<<endl;
    flag=-9.0;
}

```

```
    for (int ii=1;ii<=npointout;ii+=1){
        Tout(jj,ii-1)=T(Lout(ii-1),L1+(Lout(ii-1)-
L)*tan(beta));
    }

    Tmax(jj)=T[].max;
    if (Tmax(jj)>=TTMAX){
        TTMAX=Tmax(jj);
        TMAX=T;
    }

    TTT=0.0*(region==ww)+0.0*(region==ccz)+T*(region==cu);
    TmaxCU(jj)=TTT[].max;
    if(TmaxCU(jj)>700.0){
        ff02 << "Warning! Cu temperature exceeds
properties range of 700 C. Tile number " <<jj<<" T_Cu= "<<
TmaxCU(jj)<<" C" <<endl;
        flag=-9.0;
    }
    TTT=T*(region==ww)+0.0*(region==ccz)+0.0*(region==cu);
    TmaxW(jj)=TTT[].max;
    if(TmaxW(jj)>3200.0){
        ff02 << "Warning! W temperature exceeds properties
range of 3200 C. Tile number " <<jj<<" T_W= "<< TmaxW(jj)<<"
C" <<endl;
        flag=-9.0;
    }
    TTT=0.0*(region==ww)+T*(region==ccz)+0.0*(region==cu);
    TmaxCCZ(jj)=TTT[].max;
    if(TmaxCCZ(jj)>433.0){
        ff02 << "Warning! CuCrZr temperature exceeds
properties range of 433 C. Tile number " <<jj<<" T_CCZ= "<<
TmaxCCZ(jj)<<" C" <<endl;
        flag=-9.0;
    }
    plot(T,fill=1,value=1);
    if (jj==iplot){
        string name = "Temp2D_VT_"+sinout+"_" + (jj+1) +
".eps";
        plot(T,fill=1,value=1,ps=name);
        iplot=iplot+nplots;
    }
}
```

```

} //end loop on monoblock length
string name = "Temp2D_VT_" + sinout + "_" + Ntile + ".eps";
plot(T, fill=1, value=1, ps=name);
name = "Temp2D_VT_" + sinout + "_MAX.eps";
plot(TMAX, fill=1, value=1, ps=name);
ofstream ff1("T_CCZ_VT_" + sinout + ".dat");
ofstream ff2("T_Cu_VT_" + sinout + ".dat");
ofstream ff3("T_W_VT_" + sinout + ".dat");
ofstream ff4("T_water_VT_" + sinout + ".dat");
ofstream ff6("Tout_VT_" + sinout + ".dat");
ofstream ff7("Q_VT_" + sinout + ".dat");
ofstream ff8("XT_VT_" + sinout + ".dat");
ifstream ff9("X_tg" + sinout + ".dat");
real X;
TTMAX=0.0;
for(int jj=0; jj<Ntile; jj+=1) {
    ff1<<TmaxCCZ(jj)<<endl;
    ff2<<TmaxCU(jj)<<endl;
    ff3<<TmaxW(jj)<<endl;
    if (TmaxW(jj)>=TTMAX) {
        TTMAX=TmaxW(jj);
        IMAX=jj;
    }
    ff4<<Twater(jj)<<endl;
    for (int kk=1; kk<npointout; kk+=1) {
        ff6<<Tout(jj, kk-1)<<" ";
    }
    ff6<<Tout(jj, npointout-1)<<endl;
    ff7<<Qreal(jj)<<endl;
    ff9>>X;
    ff8<<X<<"          "<<TmaxCCZ(jj)<<"          "<<TmaxCU(jj)<<"
    "<<TmaxW(jj)<<" "<<Qreal(jj)<<endl;
    ff9>>X; ff9>>X;
}

cout<<"Maximum W temperature is "<<TTMAX<<" C at cut number
"<<IMAX<<endl;
cout<<"Water outlet temperature is "<<Twater(Ntile-1)<<"
C"<<endl;
if(flag<0.0) {
    cout<<"WARNING! Something went wrong, please check the
file 'warning.log' for details"<<endl;
}

```

}

Appendix 3

Script of the reflector plates heat transfer model

```
//Load input data and define geometry
ifstream inputs("rps_in.dat");
real
L1,LL2,L2,L4,H1,H2,H3,H4,H5,aalpha,alpha,bbeta,beta,ttheta,t
heta,Tgap;
real Hhole,Rh,Xshadow;
int n,npointout;
real Tf;
real Ltile;
real mflow,cpw;
int inout,nplots,iplot=0;
string sinout;
inputs>>L1;
inputs>>LL2;
inputs>>L4;
inputs>>H1;
inputs>>H2;
inputs>>H3;
inputs>>H4;
inputs>>H5;
inputs>>Hhole;
inputs>>Rh;
```



```
inputs>>Ltile;
inputs>>Tgap;
inputs>>aalpha;
inputs>>bbeta;
inputs>>ttheta;
inputs>>Tf;
inputs>>mflow;
inputs>>cpw;
inputs>>n;
inputs>>npointout;          // Number of points for output
surface temperature
inputs>>inout; //defines inner (1) or outer (0)
inputs>>nplots; //2D plots are produce each nplots cuts
if (inout==1){
    sinout="i";}
else{
    if (inout==0){
        sinout="o";}
    else{
        sinout="unknown";}
}
real[int] Lout(npointout);
for (int kk=1;kk<=npointout;kk+=1){
    inputs>>Lout(kk-1);
}
L2=L1-LL2-Rh-Rh;
alpha=aalpha*pi/180.0;
beta=bbeta*pi/180.0;
theta=ttheta*pi/180.0;
Xshadow=(2*L1*tan(beta)-
Tgap*tan(theta))/(tan(beta)+tan(theta));

real Htot=H1+H2+H3+H4+H5;
//MATERIALS DATA
//Thermal conduct CuCrZr
real[int] CCZ(4);
CCZ(0)=5.257798071745529e-07;
CCZ(1)=-6.451101399520426e-04;
CCZ(2)=2.576784427085231e-01;
CCZ(3)=3.129694443435334e+02;
//Thermal cond Cu
real[int] CU(4);
```

```

CU(0)=3.085132448660574e-08;
CU(1)=-2.277103401239594e-05;
CU(2)=-6.686469397617298e-02;
CU(3)=4.021574134794590e+02;
//Thermal cond SS
real[int] SS(2);
SS(0)=1.502e-2;
SS(1)=13.98;
//Thermal cond W
real[int] WW(6);
WW(0)=2.763634292770776e-16;
WW(1)=-2.020389882180738e-12;
WW(2)=-2.780235961230648e-09;
WW(3)=4.516924060319057e-05;
WW(4)=-1.054501953718856e-01;
WW(5)=1.750724560926432e+02;

//Load heat flux profile
ifstream size1(sinout+"rp_size.dat");
ifstream www1("Wtot_"+sinout+"rp.dat");
ifstream www2("Wpls_"+sinout+"rp.dat");
int Ntile;
size1>>Ntile;
real[int] qtot(Ntile);
real[int] qpls(Ntile);
for(int nn=0;nn<Ntile;nn+=1){
    www1>>qtot(nn);
    www2>>qpls(nn);
}
real[int] Tmax(Ntile);
real[int] TmaxSS(Ntile);
real[int] TmaxCCZ(Ntile);
real[int] TmaxW(Ntile);
real[int] TmaxCU(Ntile);
real[int] Twater(Ntile);
real[int] Qreal(Ntile);
real[int] HTC(Ntile);
real[int,int] Tout(Ntile,npointout);
real TTMAX=0.0;
int IMAX=0;
real htc;

```

```

//DOMAIN GENERATION
//DOMAIN SS
border AA1 (t=0., 2.*L1) {x=t;y=0.0;}
border AA2 (t=0., H1+H2) {x=2.*L1;y=t;}
border AA3 (t=0., L2) {x=2.*L1-t;y=H1+H2;};
border AA4 (t=0., H2) {x=2.*L1-L2;y=H1+H2-t;}
border AA5 (t=2.*L1-L2, L2) {x=t;y=H1;}
border AA6 (t=H1, H2+H1) {x=L2;y=t;}
border AA7 (t=L2, 0.) {x=t;y=H1+H2;}
border AA8 (t=H1+H2, 0.) {x=0.0;y=t;}
//DOMAIN CCZ
border BB1 (t=H1+H2, H1+H2+Hhole) {x=L2;y=t;}
border
BB2 (t=pi, 0.) {x=L2+Rh+Rh*cos(t);y=H1+H2+Hhole+Rh*sin(t);}
border BB3 (t=H1+H2+Hhole, H1+H2) {x=L2+2.*Rh;y=t;}
border BB4 (t=L2+2.*Rh, 2.*L1-L2-2.*Rh) {x=t;y=H1+H2;}
border BB5 (t=H1+H2, H1+H2+Hhole) {x=2.*L1-L2-2.*Rh;y=t;}
border
BB6 (t=pi, 0.) {x=2.*L1-L2-
Rh+Rh*cos(t);y=H1+H2+Hhole+Rh*sin(t);}
border BB7 (t=H1+H2+Hhole, H1+H2) {x=2.*L1-L2;y=t;}
border BB8 (t=H1+H2, H1+H2+H3) {x=2.*L1;y=t;}
border BB9 (t=2.*L1, L1+L4) {x=t;y=H1+H2+H3;}
border BB10 (t=L1+L4, L1-L4) {x=t;y=H1+H2+H3;}
border BB11 (t=L1-L4, 0.0) {x=t;y=H1+H2+H3;}
border BB12 (t=H1+H2+H3, H1+H2) {x=0.;y=t;}
//DOMAIN Cu
border CC1 (t=H1+H2+H3, H1+H2+H3+H4) {x=2.*L1;y=t;}
border CC2 (t=2.*L1, L1+L4) {x=t;y=H1+H2+H3+H4;};
border CC3 (t=H1+H2+H3+H4, H1+H2+H3) {x=L1+L4;y=t;}
border CC4 (t=H1+H2+H3, H1+H2+H3+H4) {x=L1-L4;y=t;}
border CC5 (t=L1-L4, 0.0) {x=t;y=H1+H2+H3+H4;};
border CC6 (t=H1+H2+H3+H4, H1+H2+H3) {x=0.0;y=t;}
//DOMAIN W
border DD1 (t=H1+H2+H3+H4, Htot) {x=2.*L1;y=t;}
border DD2 (t=0.0, L1-L4) {x=2.*L1-t;y=Htot-t*tan(beta);};
border
DD3 (t=Htot-(L1-
L4)*tan(beta), H1+H2+H3+H4) {x=L1+L4;y=t;}
border
DD4 (t=H1+H2+H3+H4, Htot-(L1+L4)*tan(beta)) {x=L1-
L4;y=t;}
border
DD5 (t=0.0, L1-L4) {x=L1-L4-t;y=Htot-
(L1+L4+t)*tan(beta);};

```

```

border                                     DD6 (t=Htot-
2.*L1*tan(beta),H1+H2+H3+H4) {x=0.0;y=t;}

mesh
block=buildmesh(AA1(6*n)+AA2(n)+AA3(n)+AA4(n)+AA5(5*n)+AA6(n)
)+AA7(n)+AA8(n)+BB1(n)+BB2(n)+BB3(n)+BB4(5*n)+BB5(n)+BB6(n)+
BB7(n)+BB8(n)+BB9(3*n)+BB10(n/5)+BB11(3*n)+BB12(n)+CC1(n/2)+
CC2(3*n)+CC3(n/2)+CC4(n/2)+CC5(3*n)+CC6(n/2)+DD1(n)+DD2(3*n)
+DD3(n)+DD4(n)+DD5(3*n)+DD6(n));
plot(block,ps="domain_RP.eps");

//PROBLEM DEFINITION
fespace Vh(block,P1);
Vh T=Tf,v,Told,TTT,TMAX,HF;
Vh kCCZ,kSS,kW,kCu,K,qq;
int
cu1=block(L1/2.,H1+H2+H3+H4/2.).region,cu2=block(L1/2.+L1,H1
+H2+H3+H4/2.).region,ww1=block(L1/2.,H1+H2+H3+H4+H5/2.).regi
on,ww2=block(L1/2.+L1,H1+H2+H3+H4+H5/2.).region,ss=block(L1/
2.,H1/2.).region,ccz=block(L1/2.,H1+H2+Hhole).region;

problem
divert(T,v)=int2d(block)(K*(dx(T)*dx(v)+dy(T)*dy(v)))-
int1d(block,DD2,DD5)(qq*v)
-
int1d(block,AA4,AA5,AA6,BB1,BB2,BB3,BB4,BB5,BB6,BB7)(htc*Tf*
v)

+int1d(block,AA4,AA5,AA6,BB1,BB2,BB3,BB4,BB5,BB6,BB7)(htc*T*
v);

//Loop on the RP length
//ofstream ff00("Qwater.dat");
//ofstream ff01("Qplasma.dat");
ofstream ff02("warning_RP_"+sinout+".log");
real flag=0.0;
TTMAX=0.0;
for(int jj=0;jj<Ntile;jj+=1){
    qq=(qpls(jj)/cos(beta+alpha))*(x>=Xshadow)+(qtot(jj)-
qpls(jj));
    Qreal(jj)=qpls(jj)/cos(beta+alpha)+qtot(jj)-qpls(jj);

```

```
real err=1.0,tol=1e-6;
Told=Tf;
real qwater=0.0,qplasma=0.0;
real Twa;
//Solve pseudo transient on the single tile

while(err>tol){
    kCCZ=CCZ(0)*T^3+CCZ(1)*T^2+CCZ(2)*T+CCZ(3);
    kSS=SS(0)*T+SS(1);
    kCu=CU(0)*T^3+CU(1)*T^2+CU(2)*T+CU(3);

    kW=WW(0)*T^5+WW(1)*T^4+WW(2)*T^3+WW(3)*T^2+WW(4)*T+WW(
5);

    K=kW*(region==ww1)+kCu*(region==cul)+kW*(region==ww2)+
kCu*(region==cu2)+kSS*(region==ss)+kCCZ*(region==ccz);
    htc=(8.0833e-04*(T^3.0)-
0.4621*(T^2.0)+116.7077*T+2.4503e+04)*(T<250.0)+(2.4803e-
07*(T^5.0)-5.5720e-04*(T^4.0)+0.4769*(T^3.0)-
90.3849*(T^2.0)-1.4571e+04*T+3.8139e+06)*(T>=250.0);
    divert;
    err=sqrt(int2d(block)((T-
Told)^2))/sqrt(int2d(block)(T^2));
    Told=T;

    }//end pseudo transient
    Twater(jj)=Tf;
    HTC(jj)=int1d(block,AA4,AA5,AA6,BB1,BB2,BB3,BB4,BB5,BB
6,BB7)(htc)/int1d(block,AA4,AA5,AA6,BB1,BB2,BB3,BB4,BB5,BB6,
BB7)(1.0);

    //Evaluate next T bulk
    qwater=(-
int1d(block,AA4,AA5,AA6,BB1,BB2,BB3,BB4,BB5,BB6,BB7)(htc*Tf)
+int1d(block,AA4,AA5,AA6,BB1,BB2,BB3,BB4,BB5,BB6,BB7)(htc*T)
)*2.0*Ltile;
    //ff00<<qwater<<endl;
    //qplasma=int1d(block,DD2,DD5)(qq)*Ltile;
    //ff01<<qplasma<<endl;
    Tf=Tf+qwater/mflow/cpw;

    //Check for CHF (20 MW/m^2)
```

```

real hfmax=0.0,hflux, npoints=300, CHF=20.0e6;
HF=htc*(T-Tf);
for (int ii=0;ii<=npoints;ii+=1.0){//border AA6
    theta=H2/npoints*ii;
    hflux=HF(L2,H1+theta);
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border BB1
    theta=Hhole/npoints*ii;
    hflux=HF(L2,H1+H2+theta);
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border BB2
    theta=ii/npoints*pi/2.0;
    hflux=HF(L2+Rh+Rh*cos(pi/2.-
theta),H1+H2+Hhole+Rh*sin(pi/2.-theta));
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border BB3
    theta=Hhole/npoints*ii;
    hflux=HF(L2+2.*Rh,H1+H2+theta);
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border BB4
    theta=(2.*L1-2.*L2-4.*Rh)/npoints*ii;
    hflux=HF(L2+2.*Rh+theta,H1+H2);
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border BB5
    theta=Hhole/npoints*ii;
    hflux=HF(2.*L1-L2-2.*Rh,H1+H2+theta);
    if(hflux>=hfmax){

```

```
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border BB6
    theta=ii/npoints*pi/2.0;
    hflux=HF(2.*L1-L2-Rh+Rh*cos(pi/2.-
theta),H1+H2+Hhole+Rh*sin(pi/2.-theta));
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border BB7
    theta=Hhole/npoints*ii;
    hflux=HF(2.*L1-L2,H1+H2+theta);
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
for (int ii=0;ii<=npoints;ii+=1.0){//border AA4
    theta=H2/npoints*ii;
    hflux=HF(2.*L1-L2,H1+theta);
    if(hflux>=hfmax){
        hfmax=hflux;
    }
}
if(hfmax>=CHF){
    ff02 << "Warning! Reached critical heat flux of
"<<CHF<<" W/m^2. Tile number " <<jj<<endl;
    flag=-9.0;
}

for (int ii=1;ii<=npointout;ii+=1){
    Tout(jj,ii-1)=T(Lout(ii-1),Htot-(2.*L1-Lout(ii-
1))*tan(beta));
}

Tmax(jj)=T[].max;
if (Tmax(jj)>=TTMAX){
    TTMAX=Tmax(jj);
    TMAX=T;
}
```

```

        TTT=0.0*(region==ww1)+0.0*(region==ww2)+0.0*(region==c
u1)+0.0*(region==cu2)+0.0*(region==ccz)+T*(region==ss);
        TmaxSS(jj)=TTT[].max;
        TTT=T*(region==ww1)+T*(region==ww2)+0.0*(region==cu1)+
0.0*(region==cu2)+0.0*(region==ccz)+0.0*(region==ss);
        TmaxW(jj)=TTT[].max;
        if(TmaxW(jj)>3200.0){
            ff02 << "Warning! W temperature exceeds properties
range of 3200 C. Tile number " <<jj<<" T_W= "<< TmaxW(jj)<<"
C" <<endl;
            flag=-9.0;
        }
        TTT=0.0*(region==ww1)+0.0*(region==ww2)+0.0*(region==c
u1)+0.0*(region==cu2)+T*(region==ccz)+0.0*(region==ss);
        TmaxCCZ(jj)=TTT[].max;
        if(TmaxCCZ(jj)>433.0){
            ff02 << "Warning! CuCrZr temperature exceeds
properties range of 433 C. Tile number " <<jj<<" T_CCZ= "<<
TmaxCCZ(jj)<<" C" <<endl;
            flag=-9.0;
        }
        TTT=0.0*(region==ww1)+0.0*(region==ww2)+T*(region==cu1
)+T*(region==cu2)+0.0*(region==ccz)+T*(region==ss);
        TmaxCU(jj)=TTT[].max;
        if(TmaxCU(jj)>700.0){
            ff02 << "Warning! Cu temperature exceeds
properties range of 700 C. Tile number " <<jj<<" T_Cu= "<<
TmaxCU(jj)<<" C" <<endl;
            flag=-9.0;
        }
        plot(T,fill=1,value=1);
        if (jj==iplot){
            string name = "Temp2D_RP_"+sinout+"_" + (jj+1) +
".eps";
            plot(T,fill=1,value=1,ps=name);
            iplot=iplot+nplots;
        }
    }//end loop on RP length

    string name = "Temp2D_RP_"+sinout+"_" + Ntile+ ".eps";
    plot(T,fill=1,value=1,ps=name);
    name = "Temp2D_RP_"+sinout+"_MAX.eps";

```

```

plot(TMAX,fill=1,value=1,ps=name);
ofstream ff1("T_CCZ_RP_"+sinout+".dat");
ofstream ff2("T_Cu_RP_"+sinout+".dat");
ofstream ff3("T_W_RP_"+sinout+".dat");
ofstream ff4("T_water_RP_"+sinout+".dat");
ofstream ff6("T_SS_RP_"+sinout+".dat");
ofstream ff7("Tout_RP_"+sinout+".dat");
ofstream ff8("Q_RP_"+sinout+".dat");
ofstream ff9("XT_RP_"+sinout+".dat");
ifstream ff10("X_"+sinout+"rp.dat");
TTMAX=0.0;
real X;
for(int jj=0;jj<Ntile;jj+=1){
    ff1<<TmaxCCZ(jj)<<endl;
    ff2<<TmaxCU(jj)<<endl;
    ff3<<TmaxW(jj)<<endl;
    ff4<<Twater(jj)<<endl;
    ff6<<TmaxSS(jj)<<endl;
    for (int kk=1;kk<npointout;kk+=1){
        ff7<<Tout(jj,kk-1)<<" ";
    }
    ff7<<Tout(jj,npointout-1)<<endl;
    ff8<<Qreal(jj)<<endl;
    if (TmaxW(jj)>=TTMAX){
        TTMAX=TmaxW(jj);
        IMAX=jj;
    }
    ff10>>X;
    ff9<<X<<"          "<<TmaxCCZ(jj)<<"          "<<TmaxCU(jj)<<"
    "<<TmaxW(jj)<<" "<<Qreal(jj)<<endl;
    ff10>>X;ff10>>X;
}
cout<<"Maximum W temperature is "<<TTMAX<<" C at cut number
"<<IMAX<<endl;
cout<<"Water outlet temperature is "<<Twater(Ntile-1)<<"
C"<<endl;
if(flag<0.0){
    cout<<"WARNING! Something went wrong, please check the
file 'warning.log' for details"<<endl;
}

```